



UFMT

UNIVERSIDADE FEDERAL DE MATO GROSSO
INSTITUTO DE COMPUTAÇÃO
COORDENAÇÃO DE ENSINO DE GRADUAÇÃO EM
CIÊNCIA DA COMPUTAÇÃO

**ANÁLISE DE DESEMPENHO DE SISTEMAS DE
ARQUIVOS COM VOLUMES PERSISTENTE NO
KUBERNETES COM NFS, GLUSTERFS E CEPHFS**

ANTHONY RICARDO RODRIGUES REZENDE

CUIABÁ - MT

2026



UNIVERSIDADE FEDERAL DE MATO GROSSO
INSTITUTO DE COMPUTAÇÃO
COORDENAÇÃO DE ENSINO DE GRADUAÇÃO EM
CIÊNCIA DA COMPUTAÇÃO

ANÁLISE DE DESEMPENHO DE SISTEMAS DE ARQUIVOS COM VOLUMES PERSISTENTE NO KUBERNETES COM NFS, GLUSTERFS E CEPHFS

ANTHONY RICARDO RODRIGUES REZENDE

Orientador: Prof. Dr. Roberto Benedito de Oliveira Pereira

Monografia apresentada ao Curso de Ciência da Computação, do Instituto de Computação da Universidade Federal de Mato Grosso, como requisito para obtenção do título de Bacharel em Ciência da Computação

CUIABÁ - MT

2026

*Este trabalho é dedicado a todos que um dia
sonharam em ser tornar cientistas.*

AGRADECIMENTOS

A Deus, pela força e discernimento concedidos ao longo desta jornada.

Aos meus familiares, amigos e namorada, em especial meus pais e meu irmão, por todo o apoio, paciência e incentivo incondicional durante os momentos mais desafiadores da graduação e da elaboração deste trabalho.

Ao meu orientador, Prof. Roberto B. de Oliveira Pereira, pela confiança, pelo direcionamento técnico exemplar e pelas valiosas sugestões que foram cruciais para a condução desta pesquisa e para o meu crescimento acadêmico.

A todo o corpo docente do Instituto de Computação da Universidade Federal de Mato Grosso, pelos ensinamentos compartilhados que construíram a base de conhecimento necessária para a realização deste projeto.

Aos incontáveis desenvolvedores e membros da comunidade de código aberto, cujas ferramentas, como o Kubernetes, o Linux e tantas outras bibliotecas, formaram a base tecnológica sobre a qual este trabalho foi construído.

A todos que, direta ou indiretamente, contribuíram para a conclusão deste trabalho, meus mais sinceros agradecimentos.

RESUMO

A orquestração de contêineres em infraestruturas *on-premises* impõe desafios críticos à persistência de dados, especialmente para aplicações *stateful* que demandam acesso compartilhado de leitura e escrita por múltiplos *Pods* (*ReadWriteMany* (RWX)). Este trabalho avalia quantitativamente o desempenho e a viabilidade técnica de três arquiteturas de armazenamento distintas integradas ao Kubernetes: NFS (centralizado), GlusterFS (hiperconvergente) e CephFS (orquestrado via Rook). A metodologia adotada baseia-se em pesquisa experimental, submetendo um cluster de prova de conceito a testes de estresse de escrita sequencial com cargas de trabalho variáveis. Os resultados evidenciaram que o NFS ofereceu o melhor desempenho para cargas leves (pico de 630 MB/s), mas sofreu uma degradação severa de aproximadamente 68% na vazão ao processar arquivos maiores (7 GB), devido à saturação de *buffers*. O GlusterFS apresentou desempenho intermediário, com maior variação entre cenários. Em contrapartida, o CephFS demonstrou estabilidade superior, mantendo uma vazão constante média na faixa de 150-160 MB/s em todos os cenários, ainda que com maior *overhead* de recursos computacionais. Conclui-se que a escolha da tecnologia ideal exige um compromisso estratégico entre a simplicidade e velocidade inicial do NFS versus a robustez e previsibilidade do CephFS para cargas intensivas.

Palavras-chave: Kubernetes. Armazenamento Distribuído. NFS. CephFS. GlusterFS. ReadWriteMany.

SUMÁRIO

1	INTRODUÇÃO	1
1.1	Objetivo Geral	2
1.2	Objetivos Específicos	2
1.3	Justificativa	3
1.4	Metodologia	4
1.5	Estudo de Caso	4
2	FUNDAMENTAÇÃO TEÓRICA	6
2.1	Virtualização de Servidores	6
2.1.1	Contexto Histórico	6
2.1.2	Funcionamento	7
2.1.3	Tipos de Virtualização	7
2.1.3.1	Hipervisor Tipo 1 (Bare-Metal)	8
2.1.3.2	Hipervisor Tipo 2 (Hosted)	9
2.2	Tecnologia de Contêineres	10
2.2.1	Contexto Histórico	10
2.2.2	Definição e Arquitetura	11
2.2.3	Diferenças entre Contêineres e Máquinas Virtuais	12
2.2.4	Plataforma Docker	13
2.2.5	Docker e o Padrão OCI (<i>Open Container Initiative</i>)	15
2.3	Kubernetes: Orquestração de Contêineres	16
2.3.1	Visão Histórica	16
2.3.2	Arquitetura do Kubernetes	17
2.3.2.1	O Control Plane (Camada de Gerenciamento)	18
2.3.2.2	Os Worker Nodes (Nós de Processamento)	19
2.3.3	Objetos Fundamentais do Kubernetes	20
2.4	Persistência de Dados no Kubernetes	21
2.4.1	PersistentVolume (PV)	21
2.4.2	Provisionamento de PersistentVolumes	22
2.4.3	PersistentVolumeClaim (PVC)	23
2.4.4	A Relação entre PV e PVC: O Processo de <i>Binding</i>	23
2.4.5	Modos de Acesso (Access Modes)	24
2.4.6	O Fluxo de Provisionamento e Consumo de Volumes	24
2.4.7	O Desafio das Aplicações Stateful	26

2.5	Soluções de Armazenamento para Ambientes On-Premises	26
2.5.1	NFS (Network File System)	27
2.5.1.1	Integração com Kubernetes	27
2.5.1.2	Principais Casos de Uso e Características	27
2.5.2	GlusterFS	27
2.5.2.1	Integração com Kubernetes	28
2.5.2.2	Principais Casos de Uso e Características	28
2.5.3	Ceph	28
2.5.3.1	Integração com Kubernetes	28
2.5.3.2	Principais Casos de Uso e Características	29
2.6	Trabalhos Relacionados	29
3	MATERIAIS E MÉTODOS	31
3.1	Tipo de Pesquisa	31
3.2	Limites de Estudo	32
3.3	Ambiente de Desenvolvimento	32
3.3.1	Infraestrutura	32
3.3.2	Hardware	33
3.3.2.1	Host Físico (Proxmox)	33
3.3.2.2	Nós do Cluster (Máquinas Virtuais)	33
3.3.3	Software	35
3.4	Estudo de Caso	36
3.4.1	Implantação e Preparação do Armazenamento	37
3.4.2	Procedimentos de Teste de Carga (Overhead)	37
4	DESENVOLVIMENTO	40
4.1	Procedimentos de Instalação do Cluster	40
4.2	Cenário 1: Implantação de NFS	44
4.2.1	Control Plane	45
4.2.2	Worker Nodes	47
4.2.3	Configuração no Kubernetes (Provisionamento Manual de PV e PVC)	49
4.3	Validação da Persistência e Disponibilidade	52
4.3.1	Definição da Carga de Trabalho (Pod de Teste)	53
4.3.2	Teste de Escrita e Verificação Física	54
4.4	Cenário 2: Implantação de GlusterFS	55
4.4.1	Instalação e Preparação dos Nós (Worker Nodes)	56
4.4.2	Configuração do Trusted Storage Pool e Volume	57
4.4.3	Integração com Kubernetes (PV e PVC)	59
4.4.4	Validação da Persistência e Replicação	60

4.5	Cenário 3: Implantação de CephFS via Rook	61
4.5.1	Preparação dos Dispositivos de Armazenamento	62
4.5.2	Implantação do Operador Rook	62
4.5.3	Definição e Criação do Cluster Ceph	63
4.5.4	Configuração do Sistema de Arquivos (CephFS)	64
4.5.5	Configuração da Classe de Armazenamento (StorageClass)	64
4.5.6	Provisionamento de Volume (PVC) e Pré-requisitos do Cliente	64
4.5.7	Validação de Acesso Concorrente (RWX)	65
4.6	Protocolo de Testes de Desempenho	66
4.6.1	Carga de Trabalho (Workload)	66
4.6.2	Automação e Coleta de Métricas	67
4.6.3	Estratégias de Execução por Cenário	69
5	RESULTADOS	70
5.1	NFS (Network File System)	70
5.1.1	Análise de Desempenho	71
5.1.2	Consumo de Recursos	71
5.2	GlusterFS	72
5.2.1	Análise de Desempenho	72
5.2.2	Consumo de Recursos	72
5.3	CephFS	73
5.3.1	Análise de Desempenho	73
5.3.2	Consumo de Recursos	73
5.4	Análise Comparativa dos Cenários	73
6	CONCLUSÃO	75
6.1	Trabalhos Futuros	76
	REFERÊNCIAS	78

LISTA DE ILUSTRAÇÕES

Figura 1 – Arquitetura Hipervisor Bare Metal	8
Figura 2 – Arquitetura Hipervisor Hosted	9
Figura 3 – Arquitetura simplificada de um Contêiner Docker	12
Figura 4 – Componentes Docker	14
Figura 5 – Arquitetura Cluster Kubernetes	18
Figura 6 – Fluxo de Interação entre Administrador, Desenvolvedor, PV, PVC e Pod	25
Figura 7 – Cenário de Infraestrutura de Hardware	35
Figura 8 – Fluxograma do Teste de Persistência	39
Figura 9 – Cenário 01: Cluster com NFS	45
Figura 10 – Arquitetura do Cenário 02: Cluster Kubernetes com GlusterFS Hiper- convergente	56
Figura 11 – Arquitetura do Cenário 03: Cluster Kubernetes com CephFS gerenciado pelo Rook	62
Figura 12 – Fluxo esquemático de execução dos testes de desempenho	69
Figura 13 – Comparativo de Vazão Média de Escrita entre as soluções (Maior é melhor)	74

LISTA DE TABELAS

Tabela 1 – Métricas de Desempenho de Escrita - NFS	71
Tabela 2 – Métricas de Desempenho de Escrita - GlusterFS	72
Tabela 3 – Métricas de Desempenho de Escrita - CephFS	73

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
API	Application Programming Interface (Interface de Programação de Aplicações)
CephFS	Ceph File System (Sistema de Arquivos Ceph)
CI/CD	Integração/Implantação Contínua (Continuous Integration/Continuous Deployment)
CLI	Command Line Interface (Interface de Linha de Comando)
CNCF	Cloud Native Computing Foundation
CPU	Central Processing Unit (Unidade Central de Processamento)
CRI	Container Runtime Interface
CSI	Container Storage Interface
DNS	Domain Name System (Sistema de Nomes de Domínio)
GB	Gigabytes
Guest OS	Sistema Operacional Convidado
Host OS	Sistema Operacional Hospedeiro
IC	Instituto de Computação

I/O	Input/Output (Entrada/Saída)
IP	Internet Protocol (Protocolo de Internet)
JVM	Java Virtual Machine (Máquina Virtual Java)
KVM	Kernel-based Virtual Machine
LXC	Linux Containers
MB	Megabytes
NFS	Network File System
OCI	Open Container Initiative
PV	PersistentVolume (Volume Persistente)
PVC	PersistentVolumeClaim (Requisição de Volume Persistente)
RAM	Random Access Memory (Memória de Acesso Aleatório)
RBD	RADOS Block Device
REST	Representational State Transfer
ROX	ReadOnlyMany (Somente Leitura por Vários)
RWO	ReadWriteOnce (Leitura e Escrita por Um)
RWX	ReadWriteMany (Leitura e Escrita por Vários)
SO	Sistema Operacional
TI	Tecnologia da Informação
UFMT	Universidade Federal de Mato Grosso
VM	Virtual Machine (Máquina Virtual)
VMM	Virtual Machine Monitor (Monitor de Máquina Virtual)
YAML	YAML Ain't Markup Language
GI	Gibibytes

CAPÍTULO 1

INTRODUÇÃO

A computação moderna foi fortemente impactada pelo paradigma da virtualização em nível de sistema operacional, popularizado por tecnologias de contêineres como o Docker. Essa abordagem permite o empacotamento de aplicações e suas dependências em unidades isoladas e portáteis, garantindo consistência entre ambientes de desenvolvimento, teste e produção. Nesse ecossistema, o Kubernetes emergiu como a plataforma de orquestração padrão, automatizando a implantação, o escalonamento e a gestão de aplicações contêinerizadas em larga escala e tornando-se central para a operação de software.

Contudo, a filosofia de design do Kubernetes trata os *Pods*, suas menores unidades de implantação, como recursos efêmeros e descartáveis. Essa abordagem garante alta disponibilidade e resiliência para aplicações *stateless* (sem estado), pois um *Pod* com falha pode ser imediatamente destruído e substituído (The Kubernetes, 2025b). No entanto, essa mesma característica gera um desafio significativo: a necessidade de persistência de dados. Surge, então, a questão norteadora deste trabalho: Entre as soluções de armazenamento distribuído disponíveis para ambientes *on-premises*, qual oferece o melhor equilíbrio entre desempenho, estabilidade e complexidade para suportar o compartilhamento de dados entre *Pods*?

A profundidade deste desafio se manifesta em três dificuldades centrais. Primeiramente, o paradoxo entre a efemeridade dos *Pods* e a necessidade de estado em aplicações

de negócio, como bancos de dados, que dependem de dados que devem sobreviver ao ciclo de vida do contêiner (LUKŠA, 2021). Em segundo lugar, a complexidade inerente aos ambientes *on-premises*, em que, diferentemente da conveniência dos serviços gerenciados em nuvem, a responsabilidade de planejar, implementar e manter a infraestrutura de armazenamento recai inteiramente sobre a equipe local (SANTOS, 2019). Por fim, o requisito crítico do acesso compartilhado no modo *ReadWriteMany*, essencial para aplicações colaborativas, não é suportado nativamente por muitas tecnologias de armazenamento de bloco tradicionais, tornando a busca por uma solução estável um gargalo técnico relevante (Kubernetes, 2025e; IBRYAM; HUSS, 2020).

Esta monografia está organizada em seis capítulos para apresentar o desenvolvimento da pesquisa de forma coesa. O Capítulo 1, a Introdução, contextualiza o tema, delimita o problema de pesquisa e estabelece os objetivos a serem alcançados. O Capítulo 2, Fundamentação Teórica, constrói a base conceitual necessária, revisando a literatura sobre as tecnologias envolvidas. O Capítulo 3, Metodologia, descreve a abordagem experimental e o planejamento do ambiente de testes. O Capítulo 4, Desenvolvimento, detalha a implementação técnica dos três cenários de armazenamento (NFS, GlusterFS e CephFS). O Capítulo 5, Resultados, apresenta e discute os dados quantitativos obtidos nos testes de desempenho. Por fim, o Capítulo 6, Conclusão, sintetiza as descobertas, responde à questão de pesquisa e sugere trabalhos futuros.

1.1 Objetivo Geral

O objetivo geral deste trabalho é analisar e comparar o desempenho, a estabilidade e a complexidade de implementação de diferentes arquiteturas de armazenamento distribuído em um cluster Kubernetes *on-premises*, por meio de uma prova de conceito experimental, visando identificar a solução que melhor equilibra esses critérios para prover volumes persistentes com suporte a *ReadWriteMany*.

1.2 Objetivos Específicos

Para alcançar o objetivo geral proposto, foram definidos os seguintes objetivos específicos:

- Fundamentar teoricamente os mecanismos de persistência do Kubernetes, detalhando o ciclo de vida dos objetos de armazenamento (*StorageClasses*, PVs e PVCs) e os desafios de orquestração em sistemas distribuídos.

- Projetar e implementar três cenários distintos de infraestrutura de armazenamento em um cluster *on-premises*: centralizado (NFS), hiperconvergente (GlusterFS) e orquestrado via operador (CephFS via Rook).
- Desenvolver uma metodologia de testes de desempenho (*benchmarking*) para validar o suporte ao modo de acesso *ReadWriteMany* e mensurar a capacidade de escrita sequencial.
- Coletar e processar métricas quantitativas de vazão (*throughput*), latência e consumo de recursos computacionais (CPU) para cada arquitetura sob diferentes cargas de trabalho.
- Realizar uma análise comparativa dos resultados obtidos, identificando os *trade-offs* entre complexidade de implementação, estabilidade e performance de cada solução.

1.3 Justificativa

A relevância deste trabalho se fundamenta na intersecção de três grandes vetores do cenário tecnológico atual: a consolidação do Kubernetes como padrão de orquestração (Cloud Native Computing Foundation, 2023), a necessidade de modernizar aplicações legadas e as crescentes demandas por soberania de dados e autonomia de infraestrutura. Tendo estabelecido a problemática de prover persistência compartilhada em ambientes *on-premises*, a justificativa para investigar e comparar soluções para este desafio se desdobra nos seguintes pontos:

1. Relevância de Mercado: Viabilizar a Transformação Digital em Infraestruturas Próprias

O Kubernetes tornou-se uma base central da computação em nuvem moderna. No entanto, uma parcela significativa do mercado, por razões de custo, segurança ou regulamentação, mantém sua infraestrutura de TI *on-premises* (Flexera, 2024). Muitas dessas empresas possuem sistemas críticos e bancos de dados que são inerentemente *stateful* e, sem uma solução clara para a persistência de dados, encontram uma barreira intransponível para adotar a orquestração de contêineres. A justificativa central reside na necessidade de oferecer subsídios técnicos para que estas organizações possam escolher a arquitetura de armazenamento mais adequada (centralizada ou distribuída) para modernizar suas aplicações, adotando práticas de DevOps sem a necessidade de uma migração onerosa para a nuvem pública.

2. Relevância Técnica e Acadêmica: Preenchimento de uma Lacuna de Dados Comparativos

Embora a documentação do Kubernetes explique os conceitos de *Persistent Volumes* (Kubernetes, 2025e), a aplicação prática em ambientes *on-premises* com o requisito crítico de acesso compartilhado (*ReadWriteMany*) carece de análises aprofundadas de desempenho. Há uma escassez na literatura de estudos que comparem quantitativamente o impacto de diferentes arquiteturas (NFS, GlusterFS e CephFS) na latência e vazão do cluster. Este trabalho justifica-se por fornecer dados empíricos e um guia prático, permitindo que estudantes e arquitetos de infraestrutura tomem decisões baseadas em métricas reais de *benchmark*, e não apenas em teoria, documentando os *trade-offs* entre complexidade e performance.

Em resumo, a importância desta pesquisa transcende o desafio técnico de implementação. Trata-se de democratizar o acesso à orquestração moderna e fornecer um roteiro prático e analítico para uma decisão tecnológica complexa. Este estudo é, portanto, relevante por sua aplicabilidade direta no mercado e por sua contribuição para a formação de conhecimento prático e quantitativo na área de infraestrutura distribuída.

1.4 Metodologia

A metodologia adotada nesta pesquisa é aplicada, com abordagem quantitativa, e fundamentada em uma pesquisa experimental. O objetivo é comparar objetivamente diferentes soluções de armazenamento para Kubernetes *on-premises*, avaliando-as com base em métricas de performance mensuráveis. O percurso metodológico se divide em duas fases: a primeira consiste em uma rigorosa revisão bibliográfica para solidificar a fundamentação teórica; a segunda, de caráter prático, culmina na implementação de um estudo de caso comparativo, no qual as tecnologias selecionadas são submetidas a testes de *benchmark* em um ambiente controlado para a coleta de dados quantitativos.

1.5 Estudo de Caso

O estudo de caso realizado materializou a validação prática desta pesquisa, traduzindo os conceitos teóricos em uma implementação funcional. Para isso, foi configurado um cluster Kubernetes *on-premises* em um ambiente de virtualização, composto por um nó de gerenciamento (*Control Plane*) e três nós de trabalho (*Worker Nodes*), representando uma infraestrutura mínima de produção. Neste cluster, foram implementadas e comparadas as soluções de armazenamento de rede NFS, GlusterFS e CephFS, selecionadas com base em sua capacidade de suportar o modo de acesso *ReadWriteMany*.

Para validar a eficácia das arquiteturas, aplicações de teste foram implantadas e projetadas para realizar operações de escrita concorrentes, demonstrando a persistência e

o compartilhamento de dados entre múltiplos *Pods* distribuídos pelo cluster. O resultado obtido compreende a documentação de um roteiro técnico replicável e a análise dos desafios e boas práticas identificadas durante o processo.

CAPÍTULO 2

FUNDAMENTAÇÃO TEÓRICA

Esta seção apresenta, de forma breve, os principais fundamentos teóricos envolvidos no desenvolvimento deste projeto. Serão tratados conceitos essenciais que serão aprofundados ao longo do Trabalho de Conclusão de Curso.

2.1 Virtualização de Servidores

2.1.1 Contexto Histórico

A virtualização de servidores, embora seja um pilar da computação moderna, não é uma tecnologia recente. Suas origens remontam à década de 1960, período em que a IBM, de forma pioneira, desenvolveu sistemas para particionar seus computadores *mainframe*. O objetivo era permitir que múltiplos usuários e aplicações utilizassem simultaneamente os recursos de *hardware*, que na época eram extremamente caros e complexos (IBM, 2024). Essa abordagem combatia o modelo predominante de "um servidor, uma aplicação", que resultava em uma severa ociosidade da capacidade computacional e em altos custos operacionais.

Apesar de sua concepção inicial, foi somente a partir dos anos 2000 que a virtualização se disseminou em larga escala, impulsionada pela busca incessante por eficiência em *datacenters*. A capacidade de consolidar múltiplos servidores virtuais em

uma única máquina física revolucionou a gestão de infraestrutura de TI. Essa consolidação trouxe benefícios diretos como a drástica redução de custos com *hardware*, energia elétrica e espaço físico, além de proporcionar uma agilidade sem precedentes na implantação e no gerenciamento de novos serviços (IBM, 2024). Foi essa transformação que pavimentou o caminho para o desenvolvimento da computação em nuvem e das práticas de *DevOps* atuais.

2.1.2 Funcionamento

A virtualização de servidores baseia-se na criação de uma camada de abstração entre o *hardware* e os sistemas operacionais. Essa camada, conhecida como hipervisor ou *Virtual Machine Monitor* (VMM), é a peça central da tecnologia. O hipervisor tem a responsabilidade de gerenciar e alocar os recursos de *hardware* da máquina física — como CPU, memória RAM, armazenamento e rede — entre as diversas máquinas virtuais (VMs) que estão em execução sobre ele (TANENBAUM; BOS, 2016).

Cada VM é um ambiente computacional completo e isolado, contendo seu próprio sistema operacional (conhecido como sistema operacional convidado ou *guest OS*), bibliotecas e aplicações. Do ponto de vista do sistema operacional convidado e das aplicações que nele rodam, a VM se comporta como um computador físico independente. O hipervisor intercepta as instruções que as VMs enviam ao *hardware*, as traduz e as executa de forma segura no *hardware* real, garantindo que uma VM não interfira no funcionamento das outras (SILBERSCHATZ; GALVIN; GAGNE, 2018). Esse processo de isolamento e gerenciamento de recursos permite a consolidação de múltiplos servidores em um único host físico, otimizando o uso da infraestrutura e proporcionando flexibilidade operacional.

2.1.3 Tipos de Virtualização

A implementação da camada de abstração pode ocorrer de diferentes maneiras, o que dá origem a distintos tipos de arquiteturas de virtualização. A distinção mais comum é baseada em onde o hipervisor é executado em relação ao *hardware* do hospedeiro.

O hipervisor é a camada de *software* que possibilita a virtualização. Ele é responsável por criar, executar e gerenciar as máquinas virtuais. Sua função principal é emular o *hardware* que será apresentado a cada VM e coordenar o acesso concorrente aos recursos físicos subjacentes, como processador, memória e dispositivos de I/O (Entrada/Saída). A eficiência e as características do hipervisor determinam o desempenho e as capacidades da plataforma de virtualização como um todo (STALLINGS, 2018). Existem duas arquiteturas principais de hipervisores: Tipo 1 (*Bare-Metal*) e Tipo 2 (*Hosted*).

2.1.3.1 Hypervisor Tipo 1 (Bare-Metal)

O Hypervisor Tipo 1, também conhecido como *bare-metal*, é instalado e executado diretamente sobre o *hardware* do servidor físico, atuando como o próprio sistema operacional da máquina hospedeira. Nessa arquitetura, o hypervisor possui drivers próprios para interagir com os componentes de hardware e é responsável por todo o gerenciamento de baixo nível. Por não depender de um sistema operacional subjacente para intermediar o acesso aos recursos, essa abordagem oferece baixa latência, alta segurança e maior eficiência (MAUERER, 2021). A seguir, a Figura 1 ilustra os componentes dessa estrutura:

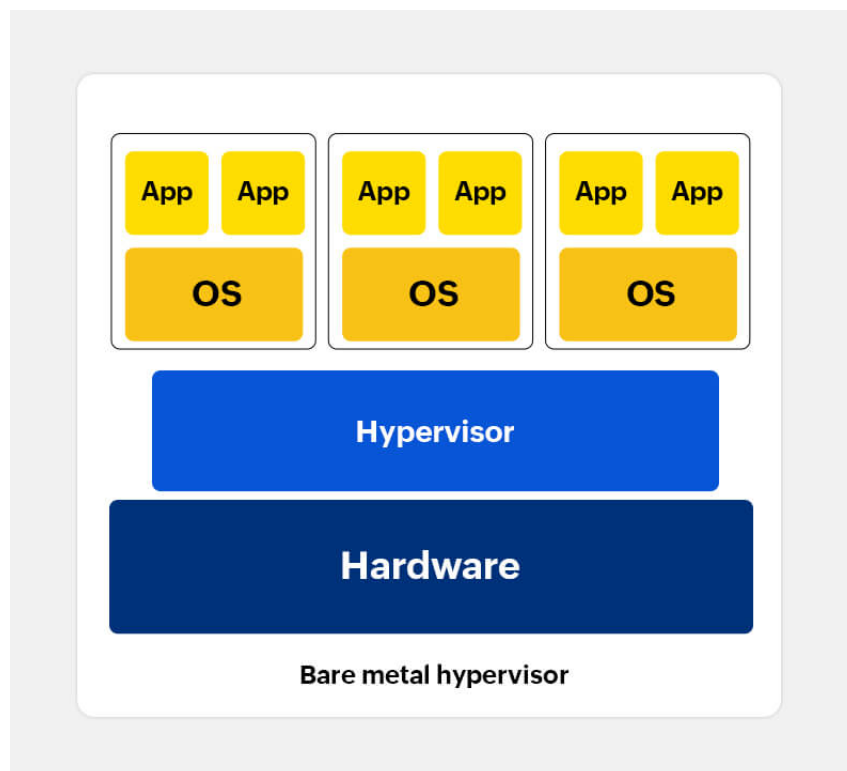


Figura 1 – Arquitetura Hypervisor Bare Metal

Fonte: (Amazon Web Services, 2024).

A Figura 1 detalha a estrutura em camadas desta arquitetura. Na base encontra-se o *hardware* da máquina. Operando diretamente sobre ele, o *hypervisor* gerencia os recursos e cria os ambientes virtuais. Acima do hypervisor, são executadas as Máquinas Virtuais, onde cada uma contém seu próprio Sistema Operacional (SO) convidado completo, juntamente com suas bibliotecas e Aplicações (App). Esta estrutura garante um forte isolamento entre as VMs, pois cada uma funciona como um servidor independente, embora compartilhem o mesmo *hardware*, gerenciado pelo hypervisor.

As máquinas virtuais são executadas sobre essa camada de virtualização, tendo um acesso mais direto aos recursos físicos. Devido a essas características, os hipervisores

Tipo 1 são a escolha padrão para ambientes de produção e *datacenters* corporativos, onde desempenho e estabilidade são críticos. Exemplos proeminentes de hipervisores Tipo 1 incluem o Proxmox VE (baseado em KVM), VMware vSphere/ESXi, Microsoft Hyper-V e Citrix Hypervisor (anteriormente XenServer) (Proxmox Server Solutions GmbH, 2024).

2.1.3.2 Hipervisor Tipo 2 (Hosted)

O Hipervisor Tipo 2, também conhecido como hosted (hospedado), opera de forma distinta do Tipo 1. Ele é executado como uma aplicação de *software* sobre um sistema operacional convencional (como Windows, macOS ou Linux), que por sua vez é instalado diretamente no *hardware*. Nessa configuração, o sistema operacional hospedeiro é o responsável primário pela gestão do *hardware*, e o hipervisor solicita recursos a ele para alocar às máquinas virtuais (IBM, 2024).

A arquitetura dessa abordagem é ilustrada na Figura 2. A estrutura em camadas é a seguinte:

- **Hardware:** A base física do sistema.
- **Host OS (Sistema Operacional Hospedeiro):** A camada de sistema operacional principal, que gerencia diretamente o *hardware*.
- **Hypervisor:** Executado como um processo dentro do SO Hospedeiro.
- **Guest OS (Sistema Operacional Convidado):** Cada máquina virtual possui seu próprio SO, que se comunica com o hipervisor para solicitar recursos.

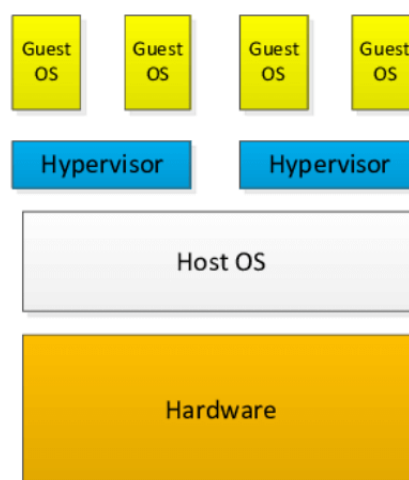


Figura 2 – Arquitetura Hipervisor Hosted

Fonte: (GOWRI, 2019).

Essa camada extra de um sistema operacional hospedeiro introduz sobrecarga (*overhead*) e maior latência, já que as requisições das VMs precisam passar por mais níveis de *software* até chegarem ao *hardware*. Em contrapartida, os hipervisores Tipo 2 são extremamente fáceis de instalar e gerenciar, pois se comportam como qualquer outro programa de computador. Tais características os tornam ideais para ambientes de desenvolvimento, testes, aprendizado e uso em desktops. Exemplos populares incluem o Oracle VM VirtualBox, VMware Workstation e Parallels Desktop (TANENBAUM; BOS, 2016).

Com a consolidação da virtualização, surgiram demandas por maior densidade de execução e menor tempo de inicialização, impulsionando abordagens mais leves baseadas no compartilhamento do kernel. Nesse contexto, a tecnologia de contêineres ganhou relevância.

2.2 Tecnologia de Contêineres

A contêinerização representa uma evolução na forma como as aplicações de *software* são desenvolvidas, empacotadas e implantadas. Ao permitir que aplicações sejam migradas entre diferentes ambientes computacionais com consistência e agilidade, os contêineres se tornaram uma tecnologia fundamental para a aceleração do desenvolvimento de *software* e para a arquitetura de sistemas modernos, como as práticas de *DevOps* e a computação nativa em nuvem (Red Hat, Inc., 2025).

2.2.1 Contexto Histórico

Ainda que os fundamentos tecnológicos para o isolamento de processos no nível do sistema operacional – notavelmente os *control groups* (cgroups) e os *namespaces* do kernel Linux – existissem há mais tempo, sua utilização era complexa e restrita a especialistas. A adoção em larga escala da tecnologia de contêineres só foi catalisada a partir do momento em que uma plataforma tornou seu uso simples e acessível aos desenvolvedores (LUKŠA, 2021).

O lançamento da plataforma Docker em 2013, como um projeto de código aberto, representou o marco decisivo que iniciou esta revolução. Inicialmente baseada na tecnologia LXC (*Linux Containers*), a solução do Docker evoluiu de forma independente, focando em simplificar a experiência de desenvolvedores e operadores ao separar as dependências das aplicações da infraestrutura (Red Hat, Inc., 2025; Docker, Inc., 2025).

O sucesso da abordagem do Docker impulsionou a padronização da tecnologia. Em junho de 2015, o Docker doou a especificação do formato de imagem e o código de tempo de execução para a *Open Container Initiative* (OCI), com o objetivo de estabelecer

um padrão para o ecossistema. Seguindo essa evolução, em 2017, o Docker doou o projeto containerd, um runtime de contêiner padrão da indústria, para a *Cloud Native Computing Foundation* (CNCF). A ampla adoção no ecossistema Linux levou a uma parceria com a Microsoft, que expandiu a funcionalidade dos contêineres Docker para o ambiente Windows Server (Docker, Inc., 2025).

2.2.2 Definição e Arquitetura

Um contêiner é uma unidade padrão de *software* que empacota uma aplicação, seu código e todas as suas dependências — como bibliotecas, ferramentas de sistema e configurações de tempo de execução (runtime) — em um único pacote executável e isolado (Docker, Inc., 2025; Red Hat, Inc., 2025). Esse empacotamento garante que a aplicação seja executada de forma rápida e confiável, independentemente do ambiente computacional, seja ele de desenvolvimento, teste ou produção.

A arquitetura dos contêineres se diferencia fundamentalmente da virtualização tradicional. Em vez de virtualizar o *hardware*, os contêineres virtualizam as aplicações. Vários contêineres podem ser executados em uma mesma máquina e compartilham o kernel do sistema operacional do hospedeiro. Cada contêiner é executado como um processo isolado no espaço de usuário, o que os torna extremamente leves e eficientes (Red Hat, Inc., 2025). A Figura 3 ilustra o funcionamento de contêineres Docker:

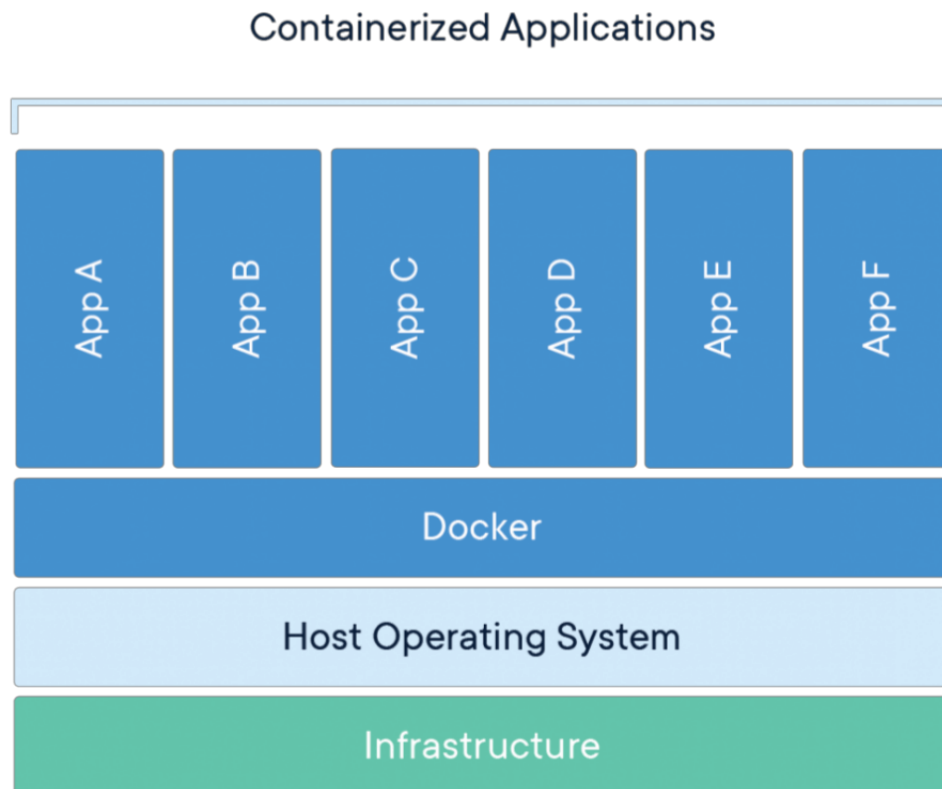


Figura 3 – Arquitetura simplificada de um Contêiner Docker

Fonte: (Docker, Inc., 2025).

A Figura 3 detalha que o processo se inicia com uma imagem de contêiner, que é um pacote leve, autônomo e executável. Em tempo de execução, essa imagem se torna um contêiner quando executada por um *runtime* de contêiner. Por compartilharem o kernel do SO, não há a necessidade de um sistema operacional completo para cada aplicação, resultando em maior eficiência e menor consumo de recursos do servidor (Docker, Inc., 2025).

2.2.3 Diferenças entre Contêineres e Máquinas Virtuais

Embora contêineres e máquinas virtuais (VMs) ofereçam benefícios semelhantes de isolamento e alocação de recursos, eles funcionam de maneiras distintas devido à camada que virtualizam (Docker, Inc., 2025).

As Máquinas Virtuais são uma abstração do *hardware*, projetadas para emular um sistema operacional completo dentro de um sistema operacional *host*. Um *software* chamado *hypervisor* permite que múltiplas VMs sejam executadas em uma única máquina física. Cada VM, no entanto, precisa incluir uma cópia integral de um sistema operacional com seu próprio *kernel*, além da aplicação e de todas as suas bibliotecas e binários. Em-

bora essa abordagem proporcione um alto nível de isolamento, ela gera uma sobrecarga significativa de recursos. Como resultado, as VMs ocupam um espaço de armazenamento considerável, tipicamente na ordem de *gigabytes* (GB), e apresentam um tempo de inicialização mais lento em comparação com tecnologias de contêineres (Docker, Inc., 2025; Red Hat, Inc., 2025).

Os Contêineres, por outro lado, são uma abstração na camada da aplicação. Eles compartilham o *kernel* do sistema operacional do hospedeiro e empacotam apenas o código da aplicação e suas dependências. Como resultado, são muito mais leves — as imagens de contêineres frequentemente medem dezenas de *megabytes* (MB). Isso permite que uma única máquina hospedeira execute um número maior de contêineres em comparação com VMs, levando a uma eficiência superior e à redução de custos com servidores e licenciamento. A natureza dos contêineres, como processos isolados, também lhes confere portabilidade e agilidade (Docker, Inc., 2025).

Apesar de suas diferenças, é possível utilizar as duas tecnologias em conjunto, onde os contêineres são executados dentro de máquinas virtuais, combinando o isolamento de *hardware* das VMs com a flexibilidade e a eficiência dos contêineres (Docker, Inc., 2025; Red Hat, Inc., 2025).

2.2.4 Plataforma Docker

Embora o termo "Docker" seja frequentemente usado como sinônimo de contêiner, é mais preciso defini-lo como uma plataforma de *software* completa que simplifica todo o ciclo de vida dos contêineres. A plataforma consiste em um conjunto de ferramentas e serviços que permitem aos desenvolvedores e operadores de sistemas criar, distribuir e executar aplicações de forma consistente em qualquer ambiente (SANTOS, 2019). Os componentes centrais da plataforma Docker seguem esquematizados na Figura 4:

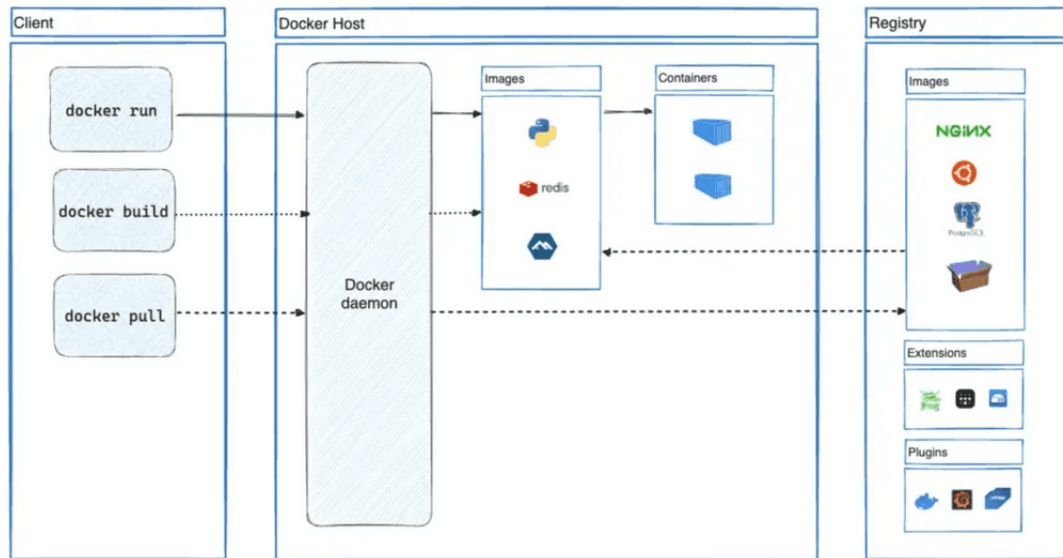


Figura 4 – Componentes Docker

Fonte: (My Nettech, 2025).

- **Docker Client (Cliente Docker):** Representado na ilustração pelos comandos `docker run`, `docker build` e `docker pull`, o Cliente Docker é a interface primária através da qual o usuário interage com o Docker. Trata-se de uma ferramenta de linha de comando (CLI) que traduz as instruções do usuário em requisições para a API REST do Docker. Quando um desenvolvedor ou administrador de sistemas executa um comando, o cliente o envia para o Docker Daemon, que efetivamente realiza a tarefa.
- **Docker Daemon:** O daemon (`dockerd`) é o "cérebro" da arquitetura Docker, um serviço de longa duração (processo em background) que executa no Docker Host. Como ilustrado, ele é o componente central que "escuta" as requisições da API vindas do Cliente Docker e gerencia todos os objetos Docker. Suas principais responsabilidades, visíveis no fluxo do diagrama, incluem:
 - Construir imagens a partir de um Dockerfile (acionado por `docker build`).
 - Gerenciar o ciclo de vida dos contêineres: criar, iniciar, parar e remover contêineres a partir de imagens (acionado por `docker run`).
 - Gerenciar o armazenamento e a distribuição de imagens: baixar imagens de um Registry (acionado por `docker pull`) e enviá-las.
 - Orquestrar volumes de dados e redes para os contêineres.
- **Imagens Docker (Docker Images):** Uma imagem é um template estático e imutável, construído em um sistema de arquivos em camadas (*layers*). Ela contém tudo o que é necessário para executar uma aplicação: o código-fonte ou binário, um runtime

(como a JVM ou o interpretador Python), bibliotecas, variáveis de ambiente e arquivos de configuração. Como a ilustração demonstra, as imagens residem tanto localmente no Docker Host (após um *build* ou *pull*) quanto remotamente em um Registry. A principal função de uma imagem é servir como base para a criação de contêineres.

- **Contêineres Docker (Docker Containers):** Se uma imagem é a "planta"(classe), um contêiner é a "casa"construída a partir dela (objeto/instância). O contêiner é a instância executável e isolada de uma imagem. Ao ser criado (docker run), o Docker adiciona uma camada superior gravável (*writable layer*) sobre as camadas imutáveis da imagem. É nesta camada que todas as modificações durante a execução do contêiner, como criação de novos arquivos ou alteração dos existentes, são armazenadas. Os contêineres são, por natureza, efêmeros e independentes, encapsulando a aplicação e garantindo que ela funcione de maneira idêntica em qualquer ambiente que possua o Docker Engine.
- **Docker Registry:** É um serviço centralizado para armazenamento e distribuição de Imagens Docker. O diagrama o representa como uma entidade externa da qual o Docker Daemon baixa (*pull*) as imagens necessárias. O Docker Hub é o registry público mais conhecido, mas organizações frequentemente utilizam registries privados (como o Harbor, Amazon ECR ou Google Artifact Registry) para armazenar imagens proprietárias de forma segura. O registry é um pilar para os fluxos de trabalho de *DevOps* e Integração/Implantação Contínua (CI/CD), permitindo o versionamento e o compartilhamento de artefatos de *software* de maneira eficiente.

2.2.5 Docker e o Padrão OCI (*Open Container Initiative*)

Com a rápida ascensão da plataforma Docker, surgiu no ecossistema de tecnologia a preocupação com a dependência de um único fornecedor e a necessidade de garantir a interoperabilidade a longo prazo. Para endereçar essa questão, em 2015, a Docker, em conjunto com a Linux Foundation e outras empresas líderes do setor, fundou a *Open Container Initiative* (OCI) (Docker, Inc., 2025).

A OCI é uma organização de governança cujo principal objetivo é criar padrões abertos e mínimos para a tecnologia de contêineres. Ao estabelecer especificações formais, a iniciativa garante que qualquer imagem de contêiner possa ser executada por qualquer motor de execução compatível, promovendo um ecossistema saudável, competitivo e inovador. As duas principais especificações mantidas pela OCI são:

1. **A Especificação de Imagem (*Image Specification*):** Define um formato padrão para o empacotamento de imagens de contêiner. Isso detalha como o sistema de

arquivos da imagem, a configuração (como variáveis de ambiente e comandos a serem executados) e os metadados devem ser estruturados. Uma imagem que adere a este padrão é considerada uma "imagem OCI".

2. **A Especificação de Tempo de Execução (*Runtime Specification*):** Descreve como um “*runtime*” de contêiner deve executar uma imagem. Ela define um ciclo de vida padrão para o contêiner (criação, inicialização, parada, exclusão) e como a configuração da imagem deve ser interpretada para criar um ambiente isolado.

A própria plataforma Docker evoluiu para se alinhar a esses padrões. O Docker Engine não executa mais os contêineres diretamente; ele delega essa tarefa a um componente chamado `containerd`, que por sua vez utiliza o `runc`, a implementação de referência da especificação de runtime da OCI. Essa adesão ao padrão OCI foi crucial para a integração do Docker com orquestradores como o Kubernetes, que dependem de uma interface padronizada de runtime de contêiner, a *Container Runtime Interface* (CRI), para gerenciar contêineres de diferentes fontes (LUKŠA, 2021).

Com a padronização e a popularização dos contêineres, tornou-se necessário orquestrar grandes quantidades de instâncias distribuídas em múltiplos servidores, o que motivou o surgimento de plataformas de orquestração.

2.3 Kubernetes: Orquestração de Contêineres

2.3.1 Visão Histórica

No início da década de 2010, a virtualização tradicional com Máquinas Virtuais era o padrão para isolar aplicações e otimizar o uso de *hardware*. No entanto, as VMs possuem uma sobrecarga significativa, pois cada uma executa um sistema operacional convidado completo.

A tecnologia de contêineres, que já existia de formas mais primitivas em sistemas Unix por décadas (como o `chroot` e posteriormente os FreeBSD Jails e Solaris Zones), prometia um isolamento mais leve, compartilhando o *kernel* do sistema operacional do host. O grande ponto de virada foi o lançamento do Docker em 2013. O Docker não inventou o contêiner, mas o tornou acessível, padronizado e imensamente popular entre os desenvolvedores. Ele simplificou o processo de empacotar uma aplicação e todas as suas dependências em uma unidade portátil e isolada.

Essa popularização gerou um novo desafio: se é fácil criar um contêiner, como gerenciar dezenas, centenas ou milhares deles, distribuídos em múltiplos servidores? Como

garantir que eles se comuniquem, se auto-recuperem em caso de falha e sejam escalados de acordo com a demanda? Esse é o problema da orquestração de contêineres.

Enquanto o mundo descobria como orquestrar contêineres, o Google já fazia isso em uma escala colossal há mais de uma década. Internamente, a empresa utilizava um sistema de gerenciamento de *cluster* chamado Borg. O Borg era responsável por agendar, iniciar, parar e gerenciar os milhões de jobs e aplicações que rodavam na infraestrutura do Google, desde o Gmail até a Pesquisa Google. Ele foi a inspiração direta e o predecessor espiritual do Kubernetes, estabelecendo muitos dos conceitos fundamentais de orquestração (VERMA et al., 2015).

A gênese do Kubernetes está diretamente ligada à ascensão do Docker e à vasta experiência do Google em gerenciamento de sistemas em larga escala. Embora o Docker, lançado em 2013, tenha popularizado e simplificado a criação de contêineres, sua arquitetura foi desenvolvida para ser executada em um único nó, o que tornava a automação e o gerenciamento de múltiplas instâncias uma tarefa complexa e manual.

Percebendo essa lacuna, um trio de engenheiros do Google — Craig McLuckie, Joe Beda e Brendan Burns — viu a oportunidade de criar uma solução. A intenção era combinar o melhor dos dois mundos: a simplicidade e o ecossistema vibrante do Docker com os robustos princípios de orquestração aprendidos ao longo de uma década com o sistema interno do Google, o Borg. O objetivo era desenvolver uma ferramenta de código aberto que fosse menos complexa que o Borg, mas que mantivesse sua essência de gerenciamento em escala, com uma interface mais amigável.

Este novo projeto foi batizado de Kubernetes, que significa "timoneiro" ou "piloto", uma analogia à sua função de guiar os "navios" (servidores) que transportam os "contêineres" (aplicações). Em uma clara homenagem ao seu predecessor, o sistema Borg do Google, o nome de código inicial do projeto foi "*Project Seven of Nine*" (Projeto Sete de Nove). *Seven of Nine* é uma personagem da série Star Trek: Voyager que é uma ex-drone Borg. Essa referência foi imortalizada no logotipo do Kubernetes, que possui sete pontas.

Anunciado publicamente em meados de 2014, o Kubernetes foi escrito do zero em Go, uma linguagem de programação que se tornava popular para o desenvolvimento de infraestrutura. A primeira versão pública já incluía funcionalidades essenciais para a orquestração, como replicação de instâncias, balanceamento de carga, descoberta de serviços e agendamento de cargas de trabalho em múltiplas máquinas (SUSNJARA, 2023).

2.3.2 Arquitetura do Kubernetes

Um *cluster* Kubernetes (Figura 5) consiste em um conjunto de máquinas de trabalho, chamadas de Nós (*Nodes*), que executam aplicações contêinerizadas, e uma

Camada de Gerenciamento (*Control Plane*). Os *worker nodes* hospedam os *Pods*, que são os componentes da carga de trabalho da aplicação. O *control plane*, por sua vez, gerencia os *worker nodes* e os *Pods* no *cluster*. Em ambientes de produção, o *control plane* geralmente é executado em múltiplos computadores e um *cluster* executa múltiplos nós, fornecendo assim tolerância a falhas e alta disponibilidade.

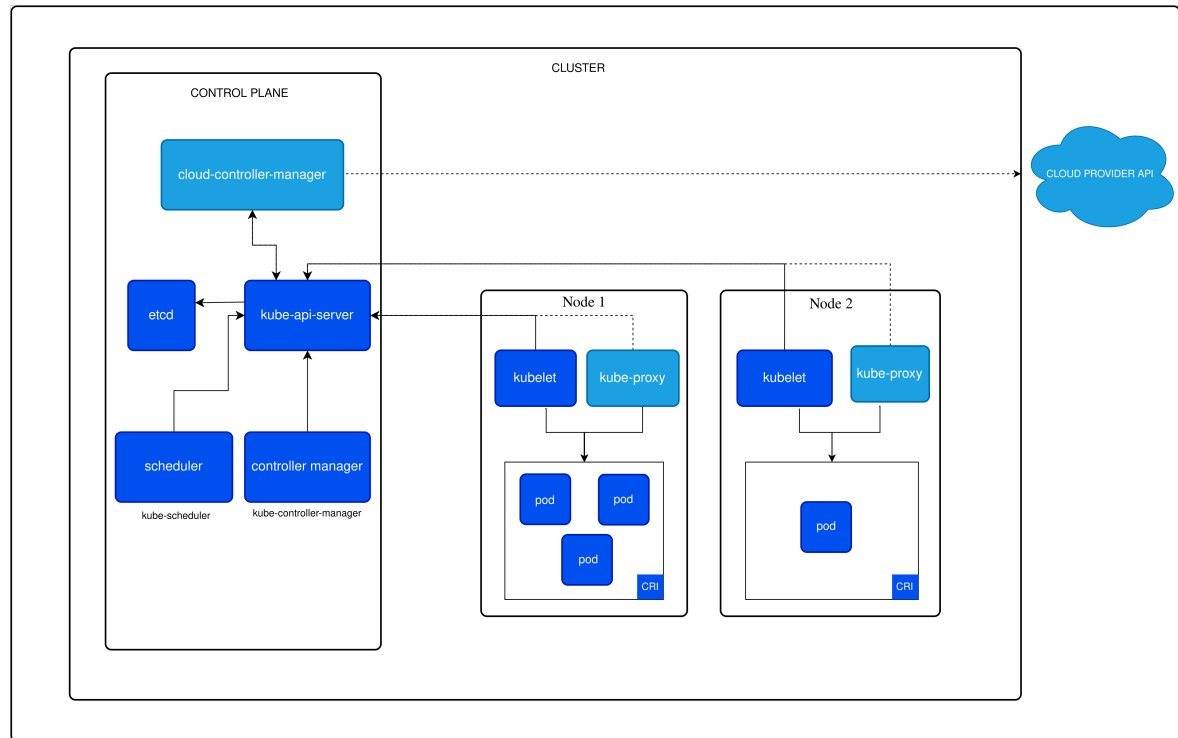


Figura 5 – Arquitetura Cluster Kubernetes

Fonte: (The Kubernetes, 2025a).

2.3.2.1 O Control Plane (Camada de Gerenciamento)

Os componentes do *control plane* são responsáveis por tomar decisões globais sobre o *cluster*, como o agendamento de cargas de trabalho, bem como por detectar e responder a eventos, como a reinicialização de um *Pod* em caso de falha.

- **kube-apiserver:** É o componente que expõe a API do Kubernetes, funcionando como o *front-end* da camada de gerenciamento. Todas as interações com o estado do *cluster*, sejam elas iniciadas por usuários ou por outros componentes, passam pelo servidor de API. Sua principal implementação, kube-apiserver, é projetada para escalar horizontalmente, permitindo que múltiplas instâncias sejam executadas para distribuir o tráfego.

- **etcd:** É um armazenamento de dados do tipo chave-valor, consistente e de alta disponibilidade. Ele funciona como o armazenamento de apoio (*backing store*) do Kubernetes para todos os dados do *cluster*, guardando a configuração e o estado de todos os recursos. É um componente crítico, sendo essencial que exista um plano de backup para seus dados.
- **kube-scheduler:** Este componente da camada de gerenciamento observa os *Pods* recém-criados que ainda não foram atribuídos a um nó e seleciona um nó para executá-los. A decisão de alocação leva em conta fatores como requisitos de recursos, restrições de *hardware* e *software*, especificações de afinidade e antiafinidade, e localidade de dados.
- **kube-controller-manager:** É o componente que executa os processos de controladores. Logicamente, cada controlador é um processo separado, mas para reduzir a complexidade, eles são compilados em um único binário e executados em um único processo. Exemplos de controladores incluem o *Node controller*, responsável por notar e responder quando um nó fica indisponível, e o *Job controller*, que observa objetos *Job* para executar tarefas pontuais.
- **cloud-controller-manager:** Este componente incorpora a lógica de controle específica de um provedor de nuvem. Ele permite vincular o *cluster* à API do provedor de nuvem, gerenciando recursos como balanceadores de carga (*load balancers*) e rotas de rede. Em ambientes *on-premises*, esse componente geralmente não é utilizado.

2.3.2.2 Os Worker Nodes (Nós de Processamento)

Os componentes do nó são executados em cada máquina de trabalho, sendo responsáveis por manter os *Pods* em execução e por fornecer o ambiente de execução do Kubernetes.

- **kubelet:** É um agente que executa em cada nó do *cluster*. Sua principal função é garantir que os contêineres descritos nos *PodSpecs* (especificações de *Pod*) estejam em execução e saudáveis. O kubelet não gerencia contêineres que não foram criados pelo Kubernetes.
- **kube-proxy:** É um *proxy* de rede executado em cada nó do *cluster*, sendo fundamental para implementar o conceito de *Service* do Kubernetes. O *kube-proxy* mantém as regras de rede nos nós, permitindo a comunicação com os *Pods* a partir de sessões de rede internas ou externas ao *cluster*. Ele utiliza a camada de filtragem de pacotes do sistema operacional, se disponível, ou encaminha o tráfego por si mesmo.

- **Container Runtime:** É o *software* responsável por executar os contêineres. O Kubernetes suporta diversos *runtimes* de contêiner, como containerd, CRI-O e qualquer outra implementação da *Container Runtime Interface* (CRI) do Kubernetes.

Compreendida a arquitetura, o próximo passo é entender os objetos que descrevem o estado desejado e o comportamento das aplicações no *cluster*.

2.3.3 Objetos Fundamentais do Kubernetes

No Kubernetes, tudo é um objeto. Os objetos do Kubernetes são entidades persistentes no sistema que representam o estado do seu *cluster*. Eles descrevem quais aplicações estão rodando, os recursos disponíveis para elas e as políticas que governam seu comportamento. O estado desejado de um objeto é definido em um campo chamado *spec*, e o status atual, observado pelo Kubernetes, é registrado no campo *status*. O Kubernetes trabalha constantemente para fazer com que o status atual corresponda à *spec* (Kubernetes, 2025b).

Esses objetos são geralmente definidos em arquivos no formato YAML e aplicados ao *cluster* através da API. Embora existam muitos tipos de objetos, os mais fundamentais para o entendimento do Kubernetes são:

- **Pod:** É a menor e mais simples unidade de implantação no Kubernetes. Um *Pod* encapsula um ou mais contêineres (como contêineres Docker), recursos de armazenamento compartilhados (Volumes), um endereço IP único de rede e opções que governam como os contêineres devem ser executados. Essencialmente, um *Pod* representa uma instância de uma aplicação em execução no *cluster*.
- **Service:** Um Service é uma abstração que define um conjunto lógico de *Pods* e uma política para acessá-los. Como os *Pods* são efêmeros e seus IPs podem mudar, o Service fornece um ponto de extremidade estável (um IP virtual e uma porta DNS) que pode ser usado para se comunicar com os *Pods* de uma aplicação. Ele também atua como um balanceador de carga, distribuindo o tráfego entre os *Pods* que ele agrupa.
- **Volume:** Um Volume é um diretório que contém dados, acessível aos contêineres em um *Pod*. Ao contrário do sistema de arquivos de um contêiner, que é efêmero, os Volumes podem ter um ciclo de vida independente do *Pod*, permitindo que os dados persistam mesmo se os contêineres forem reiniciados.
- **Namespace:** Os Namespaces são uma forma de dividir os recursos de um *cluster* em múltiplos "*clusters* virtuais". Eles fornecem um escopo para os nomes dos objetos,

permitindo que diferentes equipes ou projetos compartilhem o mesmo *cluster* físico sem interferir uns com os outros. Recursos em um namespace são isolados dos recursos em outros.

- **Deployment:** Este é um objeto de nível superior que gerencia *ReplicaSets*, que por sua vez gerenciam a criação e o escalonamento dos *Pods*. Um *Deployment* permite que você descreva o estado desejado para sua aplicação e o Kubernetes garantirá que o número correto de réplicas de *Pods* esteja sempre em execução. Mais importante, ele gerencia atualizações de forma controlada, permitindo *rollouts* (implantação de novas versões) e *rollbacks* (retorno a versões anteriores) sem tempo de inatividade.
- **ReplicaSet:** O propósito principal de um *ReplicaSet* é garantir que um número estável de réplicas de *Pods* esteja em execução em um determinado momento. Embora seja possível usá-lo diretamente, a prática recomendada é usar um *Deployment* para gerenciar *ReplicaSets*, pois o *Deployment* adiciona a funcionalidade de gerenciamento de atualizações.

Entre os objetos do Kubernetes, aqueles relacionados a armazenamento são fundamentais para aplicações que mantêm estado, pois determinam como os dados persistem além do ciclo de vida dos *Pods*.

2.4 Persistência de Dados no Kubernetes

Um dos principais desafios ao executar aplicações em contêineres é a natureza volátil de seus sistemas de arquivos. Quando um contêiner é encerrado ou falha, todas as alterações gravadas em sua camada de disco gravável são perdidas, pois o kubelet o reinicia a partir de um estado limpo da imagem original. Para aplicações não triviais, especialmente as *stateful* como bancos de dados, essa volatilidade é inaceitável, pois a perda de estado comprometeria o funcionamento dos serviços. Por isso, a persistência torna-se um requisito básico em ambientes orquestrados.

Para resolver este problema fundamental, o Kubernetes introduziu um poderoso subsistema de armazenamento que desacopla o ciclo de vida dos dados do ciclo de vida dos *Pods*. Esta arquitetura é construída sobre três conceitos principais: *Volumes*, *PersistentVolumes* (PV) com *PersistentVolumeClaims* (PVC), e *StorageClasses* (Kubernetes, 2025d).

2.4.1 PersistentVolume (PV)

Um *PersistentVolume* (PV) é um recurso no *cluster* que representa uma "fatia" de armazenamento provisionada por um administrador ou dinamicamente através de

uma *StorageClass*. Assim como um nó é um recurso de computação do *cluster*, um PV é um recurso de armazenamento (Kubernetes, 2025e; Kubernetes, 2025d). As características principais de um PV são:

- **Ciclo de Vida Independente:** Um PV possui um ciclo de vida independente de qualquer *Pod* que o utilize. Isso significa que, mesmo que o *Pod* seja destruído, o PV e os dados nele contidos persistem.
- **Abstração de Infraestrutura:** A API do PV abstrai os detalhes da implementação do armazenamento, seja ele um compartilhamento NFS, um volume iSCSI, ou um serviço de um provedor de nuvem como AWS EBS, GCE *Persistent Disk* ou Azure Disk.
- **Capacidade e Atributos:** Cada PV possui uma capacidade de armazenamento específica (ex.: 5Gi, 100Gi) e pode ter seus atributos definidos, como os modos de acesso, a classe de armazenamento e a política de recuperação (o que fazer com o volume após ser liberado).

2.4.2 Provisionamento de PersistentVolumes

Existem duas maneiras de provisionar *PersistentVolumes*:

- **Provisionamento Estático:** Um administrador de *cluster* cria manualmente uma série de PVs. Esses PVs contêm os detalhes do armazenamento real e estão disponíveis para serem requisitados por PVCs no *cluster*.
- **Provisionamento Dinâmico:** Esta é a abordagem preferida e mais flexível. Quando uma PVC é criada e nenhum PV estático atende à sua solicitação, o *cluster* pode provisionar dinamicamente um volume para aquela PVC.

Esse processo é gerenciado pelo objeto *StorageClass*. Uma *StorageClass* fornece uma maneira para os administradores descreverem as "classes" de armazenamento que oferecem. Diferentes classes podem mapear para diferentes qualidades de serviço, políticas de backup ou provedores de armazenamento. Quando uma PVC especifica uma *StorageClass*, ela instrui o *Kubernetes* a usar o provisionador associado àquela classe para criar um novo PV sob demanda, que é então imediatamente vinculado à PVC que o solicitou (Kubernetes, 2025e).

2.4.3 PersistentVolumeClaim (PVC)

Uma PVC é uma requisição de armazenamento feita por um usuário. Ela é análoga a um *Pod*: assim como um *Pod* consome recursos de um nó (CPU e memória), uma PVC consome recursos de um PV.

Uma PVC permite que um desenvolvedor ou operador de aplicação solicite armazenamento com especificações claras, sem precisar conhecer os detalhes da infraestrutura subjacente (Kubernetes, 2025e). Na definição de uma PVC, o usuário especifica:

- **Tamanho do armazenamento:** A quantidade de armazenamento necessária (ex.: 8Gi).
- **Modos de acesso:** Como o volume deve ser montado (ex.: *ReadWriteOnce*).
- **Classe de armazenamento (opcional):** O tipo de armazenamento desejado, referenciando uma *StorageClass*.

2.4.4 A Relação entre PV e PVC: O Processo de *Binding*

A interação entre PVs e PVCs segue um ciclo de vida bem definido, cujo passo central é o *binding* (vinculação).

- Um usuário cria uma *PersistentVolumeClaim* solicitando uma certa quantidade de armazenamento com um modo de acesso específico.
- O *control plane* do Kubernetes monitora constantemente novas PVCs e busca um *PersistentVolume* disponível que satisfaça os requisitos da requisição (tamanho, modo de acesso, etc.).
- Se um PV compatível é encontrado, o *cluster* realiza o bind entre o PV e a PVC. Esse vínculo é um mapeamento exclusivo de um para um, garantido pelo *ClaimRef*, uma referência bidirecional entre os dois objetos.
- Uma vez vinculada, a PVC pode ser utilizada por um *Pod* como um volume. O *Pod* simplesmente referencia o nome da PVC em sua especificação de volumes, e o Kubernetes se encarrega de montar o PV correspondente no contêiner.

Se nenhum PV compatível existir, a PVC permanecerá no estado "*pending*" até que um PV adequado seja provisionado (Kubernetes, 2025e).

2.4.5 Modos de Acesso (Access Modes)

Os modos de acesso definem como um *PersistentVolume* pode ser montado em um nó. É um atributo crucial que determina se um volume pode ser compartilhado entre múltiplos nós e se a escrita é permitida. Os modos de acesso disponíveis são:

- **ReadWriteOnce (RWO):** O volume pode ser montado como leitura e escrita por um único nó.
- **ReadOnlyMany (ROX):** O volume pode ser montado como somente leitura por vários nós simultaneamente.
- **ReadWriteMany (RWX):** O volume pode ser montado como leitura e escrita por vários nós simultaneamente.

É importante ressaltar que um volume só pode ser montado utilizando um único modo de acesso por vez, mesmo que suporte vários. A disponibilidade desses modos depende fundamentalmente da tecnologia de armazenamento subjacente. Por exemplo, um volume de bloco de um provedor de nuvem como o `awsElasticBlockStore` geralmente suporta apenas `ReadWriteOnce`, enquanto um sistema de arquivos de rede como NFS pode suportar `ReadWriteMany` (Kubernetes, 2025e).

2.4.6 O Fluxo de Provisionamento e Consumo de Volumes

A arquitetura de armazenamento persistente do Kubernetes é projetada sobre um princípio fundamental de separação de responsabilidades (*separation of concerns*). Ela distingue claramente as tarefas de provisionamento de armazenamento, geralmente a cargo de um administrador de *cluster*, das tarefas de consumo de armazenamento, realizadas por desenvolvedores ou pelas próprias aplicações. A Figura 6 ilustra essa interação.

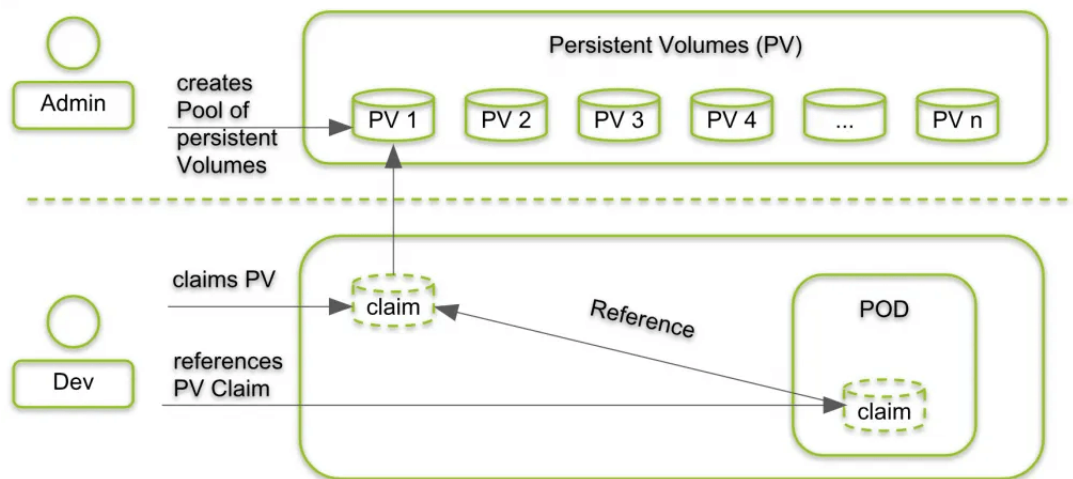


Figura 6 – Fluxo de Interação entre Administrador, Desenvolvedor, PV, PVC e Pod

Fonte: (PATEL, 2024).

O fluxo de trabalho demonstrado na imagem pode ser decomposto nas seguintes etapas conceituais:

1. Provisionamento pelo Administrador:

Na parte superior do diagrama, o Administrador do *cluster* é responsável por interagir com a infraestrutura de armazenamento subjacente (seja ela um *storage* de rede como NFS, ou um serviço de nuvem como AWS EBS). Ele cria um ou mais objetos do tipo *PersistentVolume*. Cada PV representa uma "fatia" de armazenamento real que agora está disponível para o *cluster*, formando um "pool" de recursos de armazenamento prontos para serem utilizados.

2. Requisição pelo Desenvolvedor:

O Desenvolvedor (ou o operador da aplicação), sem interagir diretamente com os PVs, define as necessidades de armazenamento da aplicação. Isso é feito através da criação de um objeto de requisição, o *PersistentVolumeClaim*, ou “reivindicação de volume persistente”. A PVC formaliza os requisitos da aplicação, especificando atributos como a capacidade de armazenamento desejada (ex.: 5Gi) e o modo de acesso necessário (ex.: *ReadWriteOnce*, que permite a montagem com leitura e escrita por um único nó).

3. Vinculação (*Binding*) pelo Kubernetes:

Após a criação da PVC, o *control plane* do Kubernetes entra em ação. Ele busca no pool de PVs um volume que atenda aos requisitos especificados na PVC. Ao encontrar um PV compatível, o Kubernetes realiza o processo de vinculação (*binding*),

conectando de forma exclusiva a PVC ao PV. A partir deste momento, aquele PV está "reivindicado" e reservado para aquela PVC específica.

4. Consumo pelo Pod:

Com a PVC no estado “vinculada” (*Bound*), o Desenvolvedor pode, finalmente, consumir o armazenamento. Na definição do *Pod*, ele não faz referência ao PV ou à infraestrutura de armazenamento real. Em vez disso, ele simplesmente declara que precisa de um volume e aponta para a PVC pelo nome. O Kubernetes automaticamente se encarrega de montar o armazenamento do PV (que está vinculado à PVC) dentro dos contêineres do *Pod*, no caminho especificado (PATEL, 2024).

Este modelo de interação oferece uma abstração poderosa. O desenvolvedor apenas precisa declarar suas necessidades de armazenamento, enquanto o administrador pode gerenciar a infraestrutura de forma centralizada e independente do ciclo de vida das aplicações, trocando provedores de armazenamento sem que as definições dos *Pods* precisem ser alteradas.

2.4.7 O Desafio das Aplicações Stateful

Aplicações *stateful*, como bancos de dados, que necessitam reter estado, enfrentam desafios intrínsecos em um ambiente de contêineres, cuja natureza é fundamentalmente efêmera. O sistema de arquivos de um contêiner é volátil, e seus dados são perdidos caso o contêiner falhe e seja reiniciado. O Kubernetes aborda essa questão através de um subsistema de armazenamento que dissocia o ciclo de vida dos dados do ciclo de vida do *Pod*. Resumidamente, o Kubernetes oferece um modelo robusto com PVs e PVCs para gerenciar o estado, mas a implementação de uma solução *stateful* resiliente exige uma análise cuidadosa das capacidades do provedor de armazenamento e de seus modos de acesso (Kubernetes, 2025e).

2.5 Soluções de Armazenamento para Ambientes On-Premises

Enquanto provedores de nuvem oferecem soluções de armazenamento totalmente integradas, a implementação de um *cluster* Kubernetes em um ambiente local (*on-premises*) exige a integração com sistemas de armazenamento externos. O Kubernetes, através de sua arquitetura flexível de volumes e, mais modernamente, da *Container Storage Interface* (CSI), permite a conexão com uma vasta gama de tecnologias de armazenamento. A escolha da solução adequada depende dos requisitos de desempenho, escalabilidade, resiliência, complexidade operacional e dos modos de acesso exigidos pelas aplicações *stateful*,

com destaque para cenários que demandam `ReadWriteMany`. A seguir, são analisadas três das soluções mais proeminentes para este cenário: NFS, GlusterFS e Ceph.

2.5.1 NFS (Network File System)

O Network File System (NFS) é um protocolo de sistema de arquivos distribuído, maduro e amplamente adotado, que permite a um cliente acessar arquivos sobre uma rede como se estivessem em um disco local (Controle Net, 2022).

2.5.1.1 Integração com Kubernetes

A integração do Kubernetes com NFS é nativa e direta. Um administrador de *cluster* pode provisionar um *PersistentVolume* de forma estática, apontando para um servidor NFS existente e um caminho de exportação específico. O Kubernetes se encarrega de montar este compartilhamento de rede no nó onde o *Pod* for agendado, disponibilizando-o para o contêiner. Embora o provisionamento estático seja o mais comum, o provisionamento dinâmico também é possível através de provisionadores externos, como o NFS CSI Driver (Kubernetes, 2025d).

2.5.1.2 Principais Casos de Uso e Características

- **Modo de Acesso Principal:** Sua principal vantagem é o suporte nativo ao modo de acesso `ReadWriteMany` (RWX), permitindo que múltiplos *Pods*, potencialmente em nós diferentes, leiam e escrevam no mesmo volume simultaneamente. É ideal para aplicações como sistemas de gerenciamento de conteúdo (CMS) ou para compartilhamento de dados entre componentes de uma aplicação.
- **Simplicidade:** É relativamente simples de configurar e manter, sendo uma solução de entrada comum para persistência de dados em Kubernetes.
- **Desvantagens:** Um servidor NFS tradicional pode se tornar um ponto único de falha (single point of failure). Além disso, sua performance para cargas de trabalho que exigem alta performance de I/O e baixa latência, como bancos de dados transacionais, pode ser inferior à de soluções de armazenamento em bloco (Controle Net, 2022).

2.5.2 GlusterFS

GlusterFS é um sistema de arquivos de rede distribuído, escalável e de código aberto. Ele agrega o espaço em disco de múltiplos servidores (chamados *bricks*) em um único namespace global, oferecendo alta disponibilidade e resiliência através da replicação e distribuição de dados (The Gluster Community, 2025).

2.5.2.1 Integração com Kubernetes

O GlusterFS se integra de forma robusta com o Kubernetes, especialmente para o provisionamento dinâmico de volumes. Utilizando ferramentas como o Heketi, que fornece uma API RESTful para gerenciar o ciclo de vida dos volumes GlusterFS, é possível configurar uma *StorageClass* que provisiona PVs sob demanda. Uma abordagem comum é a "hiperconvergência", onde os próprios servidores GlusterFS rodam como contêineres dentro do cluster Kubernetes, utilizando o armazenamento local dos nós.

2.5.2.2 Principais Casos de Uso e Características

- **Modo de Acesso Principal:** Assim como o NFS, o GlusterFS é excelente para o modo de acesso `ReadWriteMany` (RWX), sendo uma alternativa mais resiliente e escalável para compartilhamento de arquivos.
- **Alta Disponibilidade:** Ao eliminar o ponto único de falha, o GlusterFS garante que o armazenamento permaneça acessível mesmo que um dos servidores de armazenamento falhe.
- **Complexidade:** A configuração e a otimização de um cluster GlusterFS são consideravelmente mais complexas do que as de um servidor NFS simples, exigindo um planejamento cuidadoso da topologia de rede e dos volumes.

2.5.3 Ceph

Ceph é uma plataforma de armazenamento unificado, de código aberto e definida por *software*, projetada para oferecer excelente performance, confiabilidade e escalabilidade massiva. Diferente de outras soluções, o Ceph fornece três tipos de armazenamento em uma única plataforma: armazenamento de objetos (compatível com S3/Swift), armazenamento em bloco (RBD - RADOS Block Device) e sistema de arquivos (CephFS) (VUCIC, 2021).

2.5.3.1 Integração com Kubernetes

A integração do Ceph com o Kubernetes é considerada uma das mais completas e nativas para a nuvem, principalmente através do projeto Rook, um orquestrador de armazenamento certificado pela CNCF (Cloud Native Computing Foundation) (Rook Project, 2025). O Rook automatiza a implantação, configuração, escalonamento e atualização do Ceph dentro do próprio cluster Kubernetes. Através de drivers CSI, o Ceph pode oferecer dinamicamente tanto volumes em bloco quanto sistemas de arquivos.

2.5.3.2 Principais Casos de Uso e Características

- **Modos de Acesso Flexíveis:** O Ceph atende a múltiplos casos de uso. O armazenamento em bloco (RBD) é ideal para o modo `ReadWriteOnce` (RWO), oferecendo alta performance para cargas de trabalho como bancos de dados. Já o sistema de arquivos (CephFS) fornece suporte robusto para o modo `ReadWriteMany` (RWX).
- **Resiliência e Escalabilidade:** O Ceph é auto-reparável (*self-healing*) e auto-gerenciável. Ele distribui os dados de forma inteligente pelo cluster e se recupera automaticamente de falhas de disco ou de nós. Sua arquitetura permite escalar a capacidade e a performance de forma quase linear, simplesmente adicionando mais discos ou nós.
- **Complexidade e Recursos:** Ceph é, sem dúvida, a solução mais complexa e que demanda mais recursos das três apresentadas. No entanto, projetos como o Rook abstraem grande parte dessa complexidade, tornando-o uma opção viável e poderosa para ambientes de produção *on-premises*.

2.6 Trabalhos Relacionados

A orquestração de armazenamento para aplicações *stateful* em Kubernetes consolidou-se como um campo de pesquisa crítico, dado o descompasso entre a natureza efêmera dos contêineres e a necessidade perene de persistência de dados. Esta seção analisa trabalhos que utilizam tecnologias de armazenamento distribuído e protocolos de rede para mitigar essas limitações.

A gênese dos desafios de armazenamento em orquestradores pode ser rastreada até o trabalho seminal de (VERMA et al., 2015) sobre o sistema Borg, precursor do Kubernetes no Google. Os autores estabelecem o paradigma de que a computação deve ser descartável e altamente escalável. Contudo, essa abstração histórica da camada de armazenamento gerou lacunas técnicas que pesquisas contemporâneas buscam preencher ao analisar a integração de sistemas de arquivos distribuídos diretamente no plano de dados do orquestrador.

Nesse contexto, (FORNARI et al., 2023) realiza uma análise empírica comparando o desempenho de sistemas como CephFS e Lustre-ZFS em clusters Kubernetes. O estudo destaca que a execução de servidores de armazenamento contêinerizados em máquinas *bare metal* pode melhorar a flexibilidade e o failover, embora introduza variáveis de *throughput* que variam significativamente entre tecnologias de blocos e sistemas de arquivos distribuídos. Complementarmente, (GOMES, 2020) explora a implementação de Ceph como *Object Storage* em ambientes hiperconvergentes, ressaltando a resiliên-

cia do algoritmo CRUSH para eliminar metadados centralizados e evitar gargalos de escalabilidade.

A literatura também aborda modelos tradicionais de rede sob uma nova ótica. O estudo de Miano (MIANO, 2024) avalia o uso de volumes NFS em Kubernetes, comparando o provisionamento dinâmico (*Dynamic Provisioning*) com o compartilhamento estático de diretórios. O autor demonstra que, embora o NFS ofereça simplicidade, a gestão de permissões (UID/GID) e o acesso concorrente multiusuário representam desafios significativos de segurança e eficiência em ambientes orquestrados. Por outro lado, o uso de soluções integradas via operadores, como o Rook, permite que a infraestrutura de armazenamento seja autogerenciada e nativa da nuvem, conforme discutido por (LIMA; MEZZOMO, 2022) em seu estudo sobre aplicações práticas de Kubernetes e Docker.

Diferentemente dos estudos citados, que focam ou na teoria da orquestração ou na configuração de ferramentas isoladas, esta pesquisa realiza uma comparação controlada entre NFS, GlusterFS e CephFS, isolando variáveis específicas de latência e vazão em cenários de ReadWriteMany (RWX) para infraestruturas *on-premises*.

CAPÍTULO 3

MATERIAIS E MÉTODOS

Este capítulo apresenta os materiais e métodos utilizados para o desenvolvimento do projeto, abordando a metodologia de pesquisa, as ferramentas empregadas e o ambiente de desenvolvimento utilizado para a implementação do cluster Kubernetes com diferentes sistemas de armazenamento.

3.1 Tipo de Pesquisa

A metodologia adotada para este Trabalho de Conclusão de Curso pode ser classificada como uma pesquisa aplicada com abordagem quantitativa. A natureza aplicada se manifesta na construção de uma prova de conceito para um problema prático. A abordagem quantitativa é central, pois o foco da pesquisa reside na coleta e análise de dados numéricos e mensuráveis para comparar, de forma objetiva, o desempenho e o *overhead* de diferentes soluções de armazenamento persistente para Kubernetes em um ambiente *on-premises*.

A etapa inicial consiste em uma revisão bibliográfica e documental para fundamentar a teoria sobre a arquitetura de armazenamento do Kubernetes (PV, PVC, CSI) e as soluções a serem comparadas (NFS, GlusterFS e Ceph), com base em documentação oficial, livros e artigos científicos.

3.2 Limites de Estudo

Para garantir o foco e a viabilidade da pesquisa dentro do prazo estipulado, este trabalho se aterá aos seguintes limites:

- **Ambiente:** O estudo será restrito a um ambiente *on-premises*, ou seja, a infraestrutura utilizada será local e não dependerá de serviços de armazenamento gerenciados por provedores de nuvem pública (como AWS EBS, Google Persistent Disk ou Azure Disk).
- **Tecnologia de Armazenamento:** A análise se concentrará em tecnologias de armazenamento de código aberto, compatíveis com o requisito de acesso *ReadWriteMany* (RWX), como NFS, GlusterFS e Ceph. A prova de conceito implementará e comparará essas soluções.
- **Foco:** O objetivo principal é validar a persistência e o compartilhamento de dados. Aspectos como otimização de performance em larga escala não serão avaliados.
- **Aplicação de Teste:** A aplicação a ser implantada no cluster será um *software* de teste, projetado especificamente para validar as operações de escrita concorrentes no volume persistente, não se tratando de uma aplicação de produção complexa.

3.3 Ambiente de Desenvolvimento

O desenvolvimento do cluster em Kubernetes foi realizado em um ambiente configurado para garantir eficiência e compatibilidade com as tecnologias escolhidas. Para isso, foram utilizados diferentes recursos de hardware e *software*, descritos nas seções a seguir.

3.3.1 Infraestrutura

A infraestrutura para a prova de conceito foi implementada em um ambiente de virtualização local (*on-premises*). Esta escolha metodológica foi estratégica por duas razões principais: primeiro, simula um cenário de *data center* corporativo, onde muitas aplicações *stateful* ainda residem; segundo, permite um controle granular e direto sobre as camadas de rede e armazenamento. Este controle é fundamental para a análise de desempenho das soluções de persistência (NFS, GlusterFS, Ceph).

A plataforma de virtualização selecionada foi o Proxmox VE, que hospeda o conjunto de máquinas virtuais (VMs) detalhadas na seção 3.3.2. Todos os nós do cluster operam em uma rede privada isolada (192.168.1.0/24), garantindo que a comunicação entre

os componentes do Kubernetes e o *back-end* de armazenamento não sofra interferência de tráfego externo. O gerenciamento do ambiente e o acesso aos nós são centralizados via Secure Shell (SSH).

3.3.2 Hardware

O ambiente de teste foi provisionado sobre uma única máquina física (Host Físico), que executa o hipervisor Proxmox VE. Esta abordagem permite o controle total sobre a alocação de recursos e simula a falha de "servidores" distintos ao desligar uma VM, o que é fundamental para testar a resiliência das soluções de armazenamento distribuído.

3.3.2.1 Host Físico (Proxmox)

A validade dos testes quantitativos de desempenho (I/O) está diretamente ligada ao hardware subjacente. A máquina host que suporta o ambiente Proxmox possui as seguintes especificações:

- **Processador (CPU):** 24 x Intel(R) Xeon(R) CPU X5670 2.93GHz (2 Sockets)
- **Memória (RAM):** 96GB DDR3-ECC
- **Armazenamento Primário (VMs):** 1 TB NVMe SSD
- **Hipervisor:** Proxmox VE 9.0.11

3.3.2.2 Nós do Cluster (Máquinas Virtuais)

O cluster é composto por 4 (quatro) máquinas virtuais (VMs), representando 4 nós de servidor distintos. A alocação de recursos foi dimensionada para atender aos requisitos mínimos recomendados para soluções como Ceph e GlusterFS, garantindo um ambiente de teste viável para todas as tecnologias comparadas.

Todos os nós compartilham uma configuração de rede comum:

- **Subnet:** 192.168.1.0/24
- **Gateway:** 192.168.1.1
- **DNS:** 8.8.8.8

A distribuição de papéis e a configuração individual de recursos são as seguintes:

- **1x Control Plane Node** (192.168.1.242): Responsável pela gerência do cluster (API Server, etcd, Scheduler).
 - **CPU:** 4 vCPUs (x86-64)
 - **Memória:** 20 GB RAM
 - **Disco (Sistema/Armazenamento):** 100 GB Local + 100 GB SSD
- **3x Worker Nodes** (192.168.1.243, 192.168.1.244, 192.168.1.245): Responsáveis por executar as aplicações (*Pods*) e prover os serviços de armazenamento distribuído (GlusterFS, Ceph).
 - **CPU:** 6 vCPUs (x86-64)
 - **Memória:** 20 GB RAM
 - **Disco (Sistema/Armazenamento):** 100 GB Local + 100 GB SSD

A utilização de três worker nodes é o requisito mínimo para formar um quórum funcional em soluções de armazenamento distribuído como Ceph e GlusterFS, permitindo testes de alta disponibilidade (HA) e resiliência de dados. O disco local de 100 GB em cada nó é utilizado pelo sistema operacional, enquanto o SSD de 100 GB é dedicado aos *backends* de armazenamento para a criação dos volumes persistentes. Abaixo segue uma imagem que ilustra a infraestrutura utilizada.

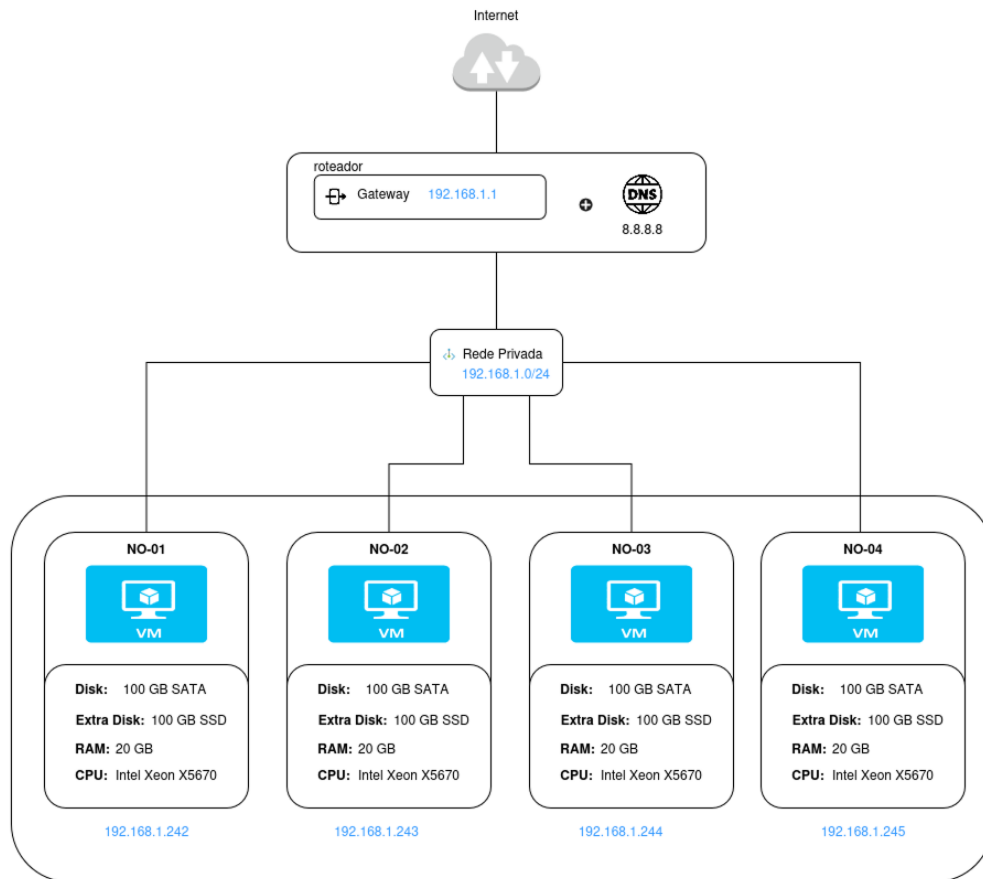


Figura 7 – Cenário de Infraestrutura de Hardware

Fonte: Autoria própria (2025).

3.3.3 Software

O ambiente foi configurado com um conjunto de *softwares* de código aberto, alinhados com as práticas padrão para a criação de um cluster Kubernetes *on-premises*. As versões foram fixadas para garantir a reprodutibilidade dos testes.

- **Sistema Operacional: Ubuntu Server 24.04 LTS** em todas as VMs. Foi escolhido por sua estabilidade, amplo suporte da comunidade e por ser uma base comum para implantações de Kubernetes em *data centers* locais.
- **Tecnologia de Contêineres: Docker Engine v26.1**. Embora o Kubernetes possa operar com outros *container runtimes*, o Docker foi selecionado por ser a ferramenta de containerização mais difundida e familiar no ecossistema de desenvolvimento.
- **Container Runtime Interface (CRI) Shim: cri-dockerd**. A partir da versão v1.24, o Kubernetes removeu o componente *dockershim* nativo, que traduzia as chamadas da API do Kubelet para o Docker Engine. Para restabelecer essa compatibilidade, o

`cri-dockerd` foi compilado e instalado em todos os nós. Esta etapa foi um requisito metodológico crucial para permitir o uso do Docker Engine como *runtime* subjacente ao cluster.

- **Orquestrador de Contêineres: Kubernetes v1.34.** A instalação foi realizada manualmente com a ferramenta `kubeadm`, que é o padrão da indústria para a inicialização (*bootstrapping*) de clusters *on-premises* e *bare-metal*. O `kubelet`, `kubeadm` e `kubect1` foram instalados e versionados em todos os nós.
- **Rede do Cluster (CNI): Calico v3.26.1.** Foi escolhido como *plugin* de CNI (Container Network Interface) por sua alta performance e, principalmente, por seu motor robusto de Políticas de Rede (*Network Policies*). Embora as políticas de rede não sejam o foco deste estudo, o Calico provê uma camada de rede confiável e bem documentada para a comunicação entre os *Pods*.
- **Soluções de Armazenamento (*ReadWriteMany*):** Para atender ao requisito de acesso compartilhado (RWX), este estudo implementará e comparará três tecnologias distintas, escolhidas por suas arquiteturas diferentes e relevância em ambientes *on-premises*:
 - **NFS (Network File System):** Utilizando o servidor NFS Kernel Server (v4.2) sobre o Ubuntu Server 24.04 LTS, hospedado na VM do *control plane*. Serve como linha de base por sua simplicidade e suporte nativo no Kubernetes, representando uma arquitetura de armazenamento centralizada (cliente-servidor).
 - **GlusterFS (Gluster File System):** Implementado na versão 11. Representa uma arquitetura de sistema de arquivos de rede distribuído e escalável, que agrega o espaço em disco dos próprios *worker nodes* para eliminar o ponto único de falha.
 - **Ceph:** Implementado na versão Reef (v18.2) e orquestrado via Rook (v1.14). O Ceph é uma plataforma de armazenamento unificado e definida por *software*, representando a solução mais complexa e robusta, que provê armazenamento em bloco (RBD) e sistema de arquivos (CephFS) de forma resiliente e escalável.

3.4 Estudo de Caso

Este capítulo detalha a execução prática do estudo de caso, descrevendo a implementação de cada cenário de armazenamento e os procedimentos de teste de *benchmark* aplicados. O objetivo desta etapa foi construir os ambientes de armazenamento sobre a infraestrutura virtualizada, validar sua funcionalidade *ReadWriteMany* (RWX) e coletar os

dados quantitativos de desempenho e *overhead*, utilizando os recursos de disco dedicados provisionados nos nós do cluster.

3.4.1 Implantação e Preparação do Armazenamento

Para garantir a isonomia dos testes e evitar a contaminação de desempenho pelo sistema operacional, foi realizada uma etapa preliminar de preparação dos discos nos nós do cluster envolvidos nos cenários de armazenamento. O disco secundário de 100 GB (`/dev/sdb`) foi preparado conforme a necessidade específica de cada tecnologia (particionado e formatado quando aplicável), garantindo que o I/O de dados da aplicação estivesse segregado do I/O do sistema operacional.

- **NFS (Network File System):** Para este cenário, foi configurado um servidor NFS centralizado na VM do *control plane*, com os *clientes* configurados nos *worker nodes*. O disco `/dev/sdb` foi particionado (`sdb1`), formatado com o sistema de arquivos `ext4` e montado no diretório `/dados`. As exportações NFS foram configuradas para permitir o acesso dos nós do Kubernetes a este diretório. No cluster, foi utilizado o *nfs-subdir-external-provisioner* para criar dinamicamente os *Persistent Volumes* (PV) mapeados para subdiretórios dentro desta montagem.
- **GlusterFS:** A implantação utilizou os discos `/dev/sdb` dos três *worker nodes* para formar um *Trusted Storage Pool*. Diferente do NFS centralizado, aqui o sistema de arquivos opera de forma distribuída. Os discos foram formatados em XFS (recomendado para GlusterFS) e montados como *bricks*. Foi criado um volume distribuído-replicado (*Distributed-Replicated Volume*) com fator de réplica 3, garantindo que cada dado gravado possua cópias síncronas em todos os nós, atendendo ao requisito de alta disponibilidade.
- **Ceph/Rook:** A implementação do Ceph foi orquestrada pelo operador Rook. Diferente das soluções anteriores, o Rook/Ceph requer acesso direto ao dispositivo de bloco para máxima performance ("*BlueStore*"). Portanto, para este cenário, o disco `/dev/sdb` foi entregue em seu estado *raw* (sem sistema de arquivos formatado previamente) para que o Rook pudesse provisionar os OSDs (*Object Storage Daemons*). Foi habilitado o CephFilesystem (CephFS) e configurado um *StorageClass* específico para permitir o modo de acesso *ReadWriteMany*.

3.4.2 Procedimentos de Teste de Carga (Overhead)

Para mensurar o desempenho de escrita e o *overhead* imposto pelas camadas de abstração de cada solução, foram realizados testes de I/O sintético. A metodologia

consistiu na escrita sequencial de arquivos de tamanhos variados, simulando cargas de trabalho intensivas.

1. **Provisionamento:** Para cada *backend* de armazenamento (NFS, GlusterFS, CephFS), foi provisionado um `PersistentVolumeClaim` (PVC) de 100 GB, montado em um *Pod* de teste baseado em Linux (Ubuntu).
2. **Execução dos Testes:** Utilizou-se a ferramenta nativa `dd` (Data Duplicator) para gerar arquivos binários preenchidos com zeros (`/dev/zero`), forçando a alocação real de blocos no disco e contornando mecanismos de cache do sistema de arquivos. O parâmetro `oflag=direct` foi utilizado para minimizar o cache de página do kernel do Linux, focando na velocidade real de gravação no *backend*.
3. **Cenários de Carga:** Foram executadas quatro baterias de 50 testes sequenciais para cada solução, registrando-se o tempo total de conclusão e a taxa de transferência (*throughput*) média. As cargas utilizadas foram as seguintes:
 - **Carga 1:** Escrita de arquivo de 1 GiB
 - **Carga 2:** Escrita de arquivo de 3 GiB
 - **Carga 3:** Escrita de arquivo de 5 GiB
 - **Carga 4:** Escrita de arquivo de 7 GiB

O procedimento passo a passo para a persistência das cargas e coleta de resultados é detalhado na Figura 8.

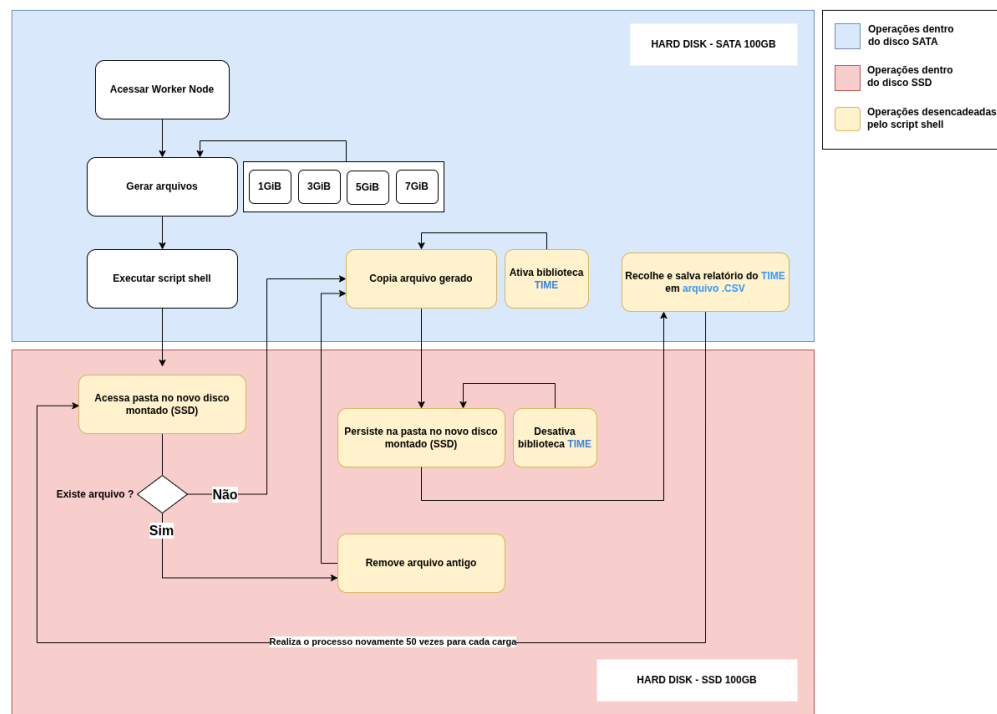


Figura 8 – Fluxograma do Teste de Persistência

Fonte: Autoria própria (2025).

O fluxo de execução, ilustrado na Figura 8, foi projetado para garantir a integridade das medições a cada ciclo. O processo inicia-se com uma etapa de verificação e limpeza: o algoritmo checa se o arquivo alvo já existe no disco de teste (SSD); caso positivo, ele é removido para evitar sobrescritas ou leituras cacheadas que poderiam enviesar os resultados.

Após a garantia de um ambiente limpo, inicia-se a etapa de medição. Nota-se, pelo diagrama, uma separação estratégica de hardware: as operações de controle de tempo (biblioteca *Time*) e o armazenamento dos logs de resultados são direcionados ao disco do sistema, enquanto a operação crítica de escrita e persistência ocorre no disco SSD dedicado ao *backend*. Essa segregação visa isolar a E/S (Entrada/Saída) do teste, impedindo que a gravação dos logs interfira na performance do disco que está sendo avaliado.

O núcleo do teste consiste na cópia do arquivo e na sua efetiva persistência no disco. Ao final, o tempo decorrido é registrado e o ciclo se repete. Conforme indicado pela seta de retorno (em vermelho), esse procedimento é iterado 50 vezes para cada carga de trabalho, garantindo uma amostra estatística relevante.

Os resultados foram tabulados comparando-se o tempo de escrita no armazenamento distribuído *versus* a performance base (*baseline*) do disco local, permitindo calcular o *overhead* percentual de cada tecnologia.

CAPÍTULO 4

DESENVOLVIMENTO

Este capítulo detalha a execução prática do estudo de caso, descrevendo a implementação de cada cenário de armazenamento e os procedimentos de teste de *benchmark* aplicados. O objetivo desta etapa foi construir os ambientes de armazenamento, validar sua funcionalidade *ReadWriteMany* (RWX) e coletar os dados quantitativos de desempenho e *overhead*, conforme definido na metodologia.

Todos os arquivos utilizados para a implantação e configuração dos serviços estão disponíveis no GitHub ¹.

4.1 Procedimentos de Instalação do Cluster

O ambiente de cluster Kubernetes, descrito no capítulo anterior, foi provisionado seguindo um roteiro técnico padronizado em todos os nós (*Control Plane* e *Workers*), garantindo a consistência do ambiente de teste. Os procedimentos foram divididos em sete etapas lógicas.

Etapa 1: Preparação do Sistema Operacional (em todos os 4 nós) Inicialmente, todos os nós baseados em Ubuntu Server 24.04 LTS foram preparados. O mecanismo de *swap* foi desabilitado permanentemente, uma vez que o *kubelet* exige que o *swap*

¹ Disponível em: <<https://github.com/pereiraR3/TCC-NFS-GLUSTER-CEPH>>

não esteja ativo para garantir o gerenciamento de memória (Kubernetes, 2025c). Isso foi feito editando o arquivo `/etc/fstab` para comentar a linha de `swap` e desativando-o imediatamente com `swapoff`.

```
1 sudo swapoff -a
2 # Edicao manual do /etc/fstab foi realizada neste ponto
3 sudo reboot
4
```

Após a reinicialização, os pacotes de dependência essenciais foram instalados, incluindo `git` e `golang-go`, este último necessário para a compilação do `cri-dockerd` na Etapa 3.

```
1 sudo apt update
2 sudo apt install -y apt-transport-https \
3     ca-certificates curl git golang-go
4
```

Etapa 2: Instalação do Docker Engine (em todos os 4 nós) O Docker Engine v26.1 foi instalado como o *container runtime*. O processo exigiu a adição do repositório oficial do Docker. Primeiramente, o diretório para as chaves GPG foi criado; em seguida, a chave GPG oficial foi baixada e suas permissões ajustadas; por fim, o repositório foi adicionado ao APT.

```
1 sudo install -m 0755 -d /etc/apt/keyrings
2 sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg
3     \
4     -o /etc/apt/keyrings/docker.asc
5 sudo chmod a+r /etc/apt/keyrings/docker.asc
6
7 echo \
8     "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt
9     /keyrings/docker.asc] \
10    https://download.docker.com/linux/ubuntu \
11    $(. /etc/os-release && echo "${UBUNTU_CODENAME:-
12    $VERSION_CODENAME}") stable" | \
13    sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Com o repositório configurado, a lista de pacotes foi atualizada e os componentes do Docker foram instalados. O usuário padrão foi adicionado ao grupo `docker` para permitir a execução de comandos sem `sudo`.

```
1 sudo apt-get update
2 sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
3     docker-buildx-plugin docker-compose-plugin
4 sudo usermod -aG docker $USER
5
```

Após este passo, uma reinicialização foi executada para aplicar as permissões de grupo.

Etapla 3: Instalação do Adaptador `cri-dockerd` (em todos os 4 nós) Para restabelecer a compatibilidade entre o Kubelet e o Docker Engine (removida do Kubernetes na v1.24), o adaptador `cri-dockerd` foi instalado. O código-fonte foi clonado, compilado localmente com `go build` e instalado em `/usr/local/bin/` (Mirantis, 2024).

```
1 git clone https://github.com/Mirantis/cri-dockerd.git
2 cd cri-dockerd
3 mkdir bin
4 go build -o bin/cri-dockerd
5 sudo install -o root -g root -m 0755 bin/cri-dockerd /usr/
6     local/bin/
```

Um serviço `systemd` (`cri-docker.service`) foi criado para gerenciar o adaptador, garantindo que ele seja iniciado após o serviço do Docker.

```
1 sudo tee /etc/systemd/system/cri-docker.service <<EOF
2 [Unit]
3 Description=CRI Docker Daemon
4 Requires=docker.service
5 After=docker.service
6
7 [Service]
8 ExecStart=/usr/local/bin/cri-dockerd
9 Restart=always
10 StartLimitInterval=0
11 RestartSec=10
12
13 [Install]
14 WantedBy=multi-user.target
15 EOF
```

16

O serviço `systemd` foi então recarregado, habilitado para iniciar no boot e iniciado.

```
1 sudo systemctl daemon-reload
2 sudo systemctl enable cri-docker.service
3 sudo systemctl start cri-docker.service
4
```

Etapa 4: Instalação dos Componentes Kubernetes (em todos os 4 nós) Os binários do Kubernetes v1.34 (`kubelet`, `kubeadm`, `kubectl`) foram instalados. Assim como o Docker, foi necessário adicionar o repositório oficial de pacotes do Kubernetes e sua chave GPG.

```
1 curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.34/deb/
   Release.key | \
2   sudo gpg --dearmor -o /usr/share/keyrings/kubernetes-
   archive-keyring.gpg
3
4 echo "deb [signed-by=/usr/share/keyrings/kubernetes-archive-
   keyring.gpg] \
5   https://pkgs.k8s.io/core:/stable:/v1.34/deb/ /" | \
6   sudo tee /etc/apt/sources.list.d/kubernetes.list
7
```

Com o repositório configurado, os binários foram instalados e o serviço `kubelet` (agente de nó) foi habilitado e iniciado.

```
1 sudo apt-get update
2 sudo apt-get install -y kubelet kubeadm kubectl
3 sudo systemctl enable kubelet && sudo systemctl start
   kubelet
4
```

Etapa 5: Inicialização do Control Plane Node No nó 192.168.1.242, o cluster foi inicializado com `kubeadm init`. O argumento `--cri-socket` foi essencial para instruir o Kubelet a usar o `cri-dockerd` instalado na etapa anterior (Kubernetes, 2025a).

```
1 sudo kubeadm init --cri-socket unix:///var/run/cri-dockerd.
   sock
2
```

Após a inicialização bem-sucedida, o arquivo de configuração `admin.conf` foi copiado para o diretório `.kube` do usuário local, permitindo a administração do cluster com `kubectl` sem privilégios de superusuário.

```
1 mkdir -p $HOME/.kube
2 sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
3 sudo chown $(id -u):$(id -g) $HOME/.kube/config
4
```

Etapas 6: Instalação da Interface de Rede (CNI) Com o Control Plane ativo, o *plugin* de rede Calico v3.26.1 foi aplicado. Esta etapa é vital para habilitar a comunicação *overlay* entre os Pods do cluster (Project Calico, 2024). A instalação foi feita aplicando-se o manifesto oficial do Calico.

```
1 kubectl apply -f \
2 https://raw.githubusercontent.com/projectcalico/calico/v3
   .26.1/manifests/calico.yaml
3
```

Etapas 7: Adição dos Worker Nodes ao Cluster Finalmente, os três *worker nodes* (VMs .243, .244, .245) foram adicionados ao cluster executando o comando `kubeadm join` obtido na saída da Etapa 5. O argumento `-cri-socket` foi novamente necessário em cada *worker*

```
1 kubeadm join <control-plane-ip>:6443 --token <token> \
2   --discovery-token-ca-cert-hash sha26:<hash> \
3   --cri-socket unix:///var/run/cri-dockerd.sock
4
```

4.2 Cenário 1: Implantação de NFS

A arquitetura lógica e física implementada neste cenário está ilustrada na Figura 9. O diagrama detalha a topologia centralizada, onde o nó *Control Plane* atua como servidor de armazenamento, disponibilizando um disco dedicado via protocolo NFS para os nós trabalhadores (*Worker Nodes*). Esta estrutura compõe a base necessária para a execução dos testes de desempenho e análise de *overhead* na persistência de dados.

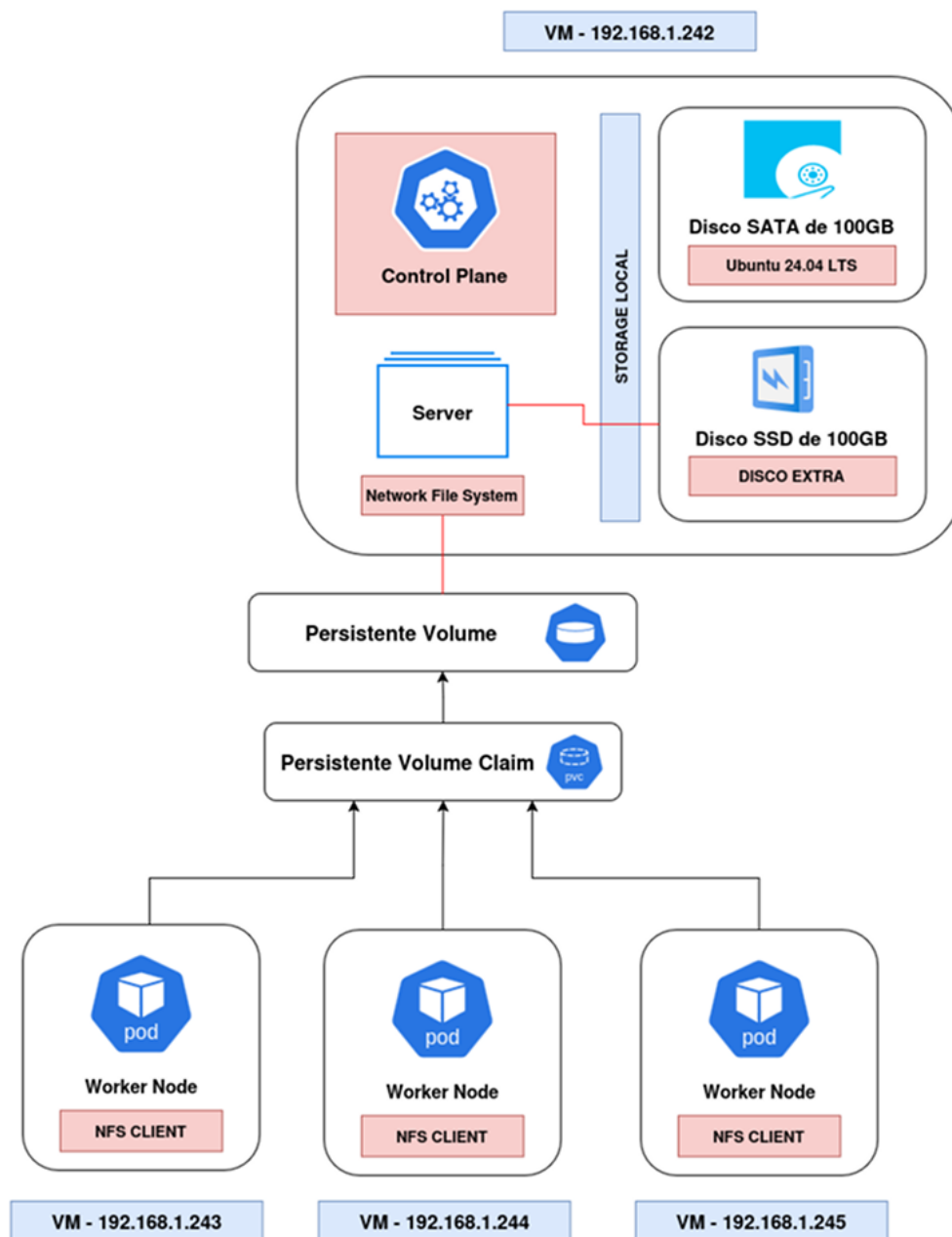


Figura 9 – Cenário 01: Cluster com NFS

Fonte: Autoria própria (2025).

4.2.1 Control Plane

A princípio, o servidor NFS foi implantado na VM do *control-plane*. Considerando que um disco secundário de estado sólido (SSD) dedicado foi anexado à máquina para armazenamento persistente e montado em `/mnt/dados`, os passos de configuração do serviço foram:

1. Instalação do servidor NFS:

```
1 sudo apt update
2 sudo apt install nfs-kernel-server -y
```

O comando `sudo apt update` sincroniza os índices de pacotes do gerenciador APT com os repositórios online. Em seguida, `sudo apt install nfs-kernel-server` instala o pacote necessário. A escolha pelo `nfs-kernel-server` se deve ao fato de ele implementar o protocolo NFS diretamente no kernel do Linux, oferecendo maior eficiência e desempenho em comparação com implementações em *userspace*.

2. Criação do diretório de exportação:

```
1 sudo mkdir -p /mnt/dados/kubedata
```

Este comando cria o diretório específico que será compartilhado na rede. A *flag* `-p` assegura a criação da estrutura de diretórios caso não exista. Optou-se por criar uma subpasta `kubedata` dentro do ponto de montagem do disco secundário (`/mnt/dados`) para melhor organização e isolamento dos dados do cluster em relação à raiz do disco.

3. Configuração de permissões:

O NFS é sensível a permissões de sistema de arquivos. Para garantir que os *pods* do Kubernetes, que podem rodar com diferentes *UIDs* (*User IDs*), consigam ler e escrever no volume, é ajustado a propriedade e as permissões do diretório de dados.

```
1 sudo chown nobody:nogroup /mnt/dados/kubedata
```

Este comando altera o dono (*owner*) e o grupo do diretório para `nobody:nogroup`. Este é o usuário e grupo padrão ao qual o NFS “mapeia” requisições de usuários não privilegiados ou do usuário *root* (quando o *root squash* está ativo).

```
1 sudo chmod 777 /mnt/dados/kubedata
```

O comando `chmod 777` concede permissão total (leitura, escrita e execução) para todos. Embora essa configuração seja permissiva demais para ambientes de produção críticos, ela é frequentemente utilizada em ambientes de desenvolvimento e prova de conceito para mitigar problemas de *access denied* decorrentes da variação de *UIDs* nos containers.

4. Exportação do diretório:

O arquivo `/etc/exports` define os diretórios compartilhados e as regras de acesso.

```
1 sudo nano /etc/exports
```

Foi adicionada a seguinte linha ao arquivo de configuração:

```
1 /mnt/dados/kubedata 192.168.1.0/24(rw, sync, no_subtree_check,
  no_root_squash) \
```

Nesta configuração, definimos o caminho exato do diretório criado e restringimos o acesso ao CIDR da rede das VMs (192.168.1.0/24), garantindo que apenas os nós do cluster tenham acesso ao volume. As opções utilizadas foram:

- `rw`: Permite operações de leitura e escrita no compartilhamento.
- `sync`: Força o servidor a confirmar a escrita dos dados no disco físico antes de responder ao cliente. Isso é vital para a integridade de dados (*data integrity*) em aplicações *stateful*, prevenindo perda de dados em caso de falha de energia, embora tenha um custo de latência.
- `no_subtree_check`: Desabilita a verificação de subárvore, melhorando a confiabilidade quando um arquivo é renomeado enquanto está aberto.
- `no_root_squash`: Opção crítica para o Kubernetes. Por padrão, o NFS converte o usuário *root* do cliente para *nobody*. A opção `no_root_squash` permite que processos rodando como *root* dentro dos containers mantenham seus privilégios de escrita no volume NFS, o que é requisito para muitas imagens de banco de dados e sistemas de arquivos distribuídos.

5. Aplicação das configurações:

```
1 sudo exportfs -arv
2 sudo systemctl restart nfs-kernel-server
```

O comando `exportfs -arv` recarrega a tabela de exportações do kernel, aplicando as novas regras imediatamente e exibindo o resultado (`verbose`). Em seguida, o reinício do serviço garante que o *daemon* esteja operando em estado limpo e pronto para conexões.

4.2.2 Worker Nodes

Para que os *Persistent Volumes* (PVs) possam ser montados e utilizados pelos Pods, cada nó (*node*) no *cluster* Kubernetes que executará cargas de trabalho (*workloads*) deve ser capaz de atuar como um cliente NFS. Isso inclui todos os *worker nodes*.

Embora o control plane geralmente seja isolado de cargas de trabalho de usuário através de *taints* (mecanismo que impede pods de serem agendados nele), a instalação do cliente NFS nesta VM também é uma boa prática para garantir uniformidade e permitir testes de montagem a partir de qualquer ponto do *cluster*.

1. Instale o cliente NFS (em todos os 3 workers):

Todos os nós que podem executar *Pods* (trabalhadores e, por precaução, o mestre) devem ter o *software* cliente instalado.

```
1 sudo apt update
2 sudo apt install nfs-common -y
```

O pacote `nfs-common` é fundamental. Ele não é o servidor (`nfs-kernel-server`), mas sim o conjunto de utilitários de cliente que provê as ferramentas necessárias (como `mount.nfs`) para que o sistema operacional possa se comunicar com um servidor NFS e montar seus compartilhamentos. Sem este pacote, o Kubelet (o principal agente do Kubernetes em cada nó) falharia ao tentar "anexar"(*attach*) o *Persistent Volume* ao *pod* que o solicitou.

2. (Opcional) Teste a montagem:

Antes de delegar a tarefa ao Kubernetes, é prudente realizar uma montagem manual em um dos *worker nodes*. Este teste valida três pontos críticos simultaneamente: 1) Conectividade de rede (se o *worker* alcança o servidor NFS), 2) Resolução de nomes/IPs e 3) As permissões de exportação (se as regras no `/etc/exports` do servidor estão corretas).

```
1 # Tenta montar o compartilhamento do servidor no /mnt local
2 sudo mount [IP_DO_SERVIDOR_NFS]:/mnt/dados/kubedata /mnt
3
4 # Verifica se funcionou
5 df -h | grep /mnt
6
7 # Se aparecer, desmonte para n o interferir com o
   Kubernetes
8 sudo umount /mnt
9
```

Neste comando (substituindo `192.168.1.242` pelo IP real do servidor NFS), `mount` é o utilitário de sistema que anexa um sistema de arquivos a um ponto no diretório local. A sintaxe `[IP]:[DIRETORIO_REMOTO]` é padrão para montagens NFS. `/mnt` é um ponto de montagem temporário padrão no Linux.

```
1 # 2. Verifica se a montagem foi bem-sucedida
2 df -h | grep /mnt
```

O comando `df -h` (*disk free - human readable*) lista todos os sistemas de arquivos montados e seu uso de espaço. Ao filtrar a saída com `grep /mnt`, buscou-se a linha

correspondente ao nosso ponto de montagem. Se o comando retornar a entrada do NFS (ex: 192.168.1.10:/mnt/dados/kubedata ... /mnt), a montagem foi bem-sucedida.

```
1 # 3. Desmonta o volume para n o interferir com o Kubelet
2 sudo umount /mnt
```

Este passo é crucial. O Kubelet gerencia o ciclo de vida completo das montagens que ele mesmo inicia. Deixar um volume montado manualmente no mesmo local que o Kubernetes utiliza pode causar conflitos e falhas. O comando `umount` remove o vínculo, garantindo que o nó esteja "limpo" para as operações do cluster.

4.2.3 Configuração no Kubernetes (Provisionamento Manual de PV e PVC)

Uma vez que o armazenamento NFS está disponível na rede, o próximo passo é integrá-lo ao Kubernetes. Este processo é realizado através da abstração de armazenamento nativa do *cluster*, que desacopla a infraestrutura de armazenamento (o "como") da sua utilização (o "o quê").

Nesta abordagem de provisionamento manual, o administrador do *cluster* deve criar explicitamente os objetos que representam o armazenamento:

- **PersistentVolume (PV):** Representa uma "peça" de armazenamento físico no *cluster*, provisionada pelo administrador. Ele é um recurso do *cluster*, assim como um Nó (*Node*) ou CPU.
- **PersistentVolumeClaim (PVC):** Representa uma "solicitação" de armazenamento feita por um usuário ou aplicação. É análogo a um *pod* solicitando CPU ou memória.

1. Criação do PersistentVolume (PV):

O PV é o objeto que "registra" o compartilhamento NFS no Kubernetes, tornando-o disponível para consumo. Foi criado um manifesto `nfs-pv.yaml` (Listing 4.1) para definir este recurso.

```
1 apiVersion: v1
2 kind: PersistentVolume
3 metadata:
4   name: nfs-pv-share
5 spec:
6   capacity:
7     storage: 100Gi
```

```
8   volumeMode: Filesystem
9   accessModes:
10    - ReadWriteMany
11   persistentVolumeReclaimPolicy: Retain
12   nfs:
13     path: /mnt/dados/kubedata
14     server: 192.168.1.242
```

Listing 4.1 – Manifesto nfs-pv.yaml

A especificação (*spec*) deste PV é fundamental para o TCC:

- **capacity.storage:** Define o tamanho total do volume (100 Gi²). No caso do NFS, o Kubernetes não impõe ativamente este limite (o limite real é o do servidor NFS). Este valor é usado primariamente pelo Kubernetes para "casar"(*bind*) o PV com um PVC que solicite uma capacidade igual ou inferior.
- **accessModes:** Este campo é crítico para aplicações *stateful*. *ReadWriteMany* (RWX) informa que este volume pode ser montado para leitura e escrita por múltiplos Nós (e, portanto, múltiplos *Pods*) simultaneamente. Esta é uma característica de sistemas de arquivos em rede (como NFS) e é essencial para arquiteturas de alta disponibilidade (HA), onde várias réplicas de um *pod* podem precisar acessar o mesmo conjunto de dados.
- **persistentVolumeReclaimPolicy:** *Retain* (Reter) Esta é a política de segurança de dados mais importante. Ela define o que acontece com o PV (e os dados físicos) quando o PVC que o utiliza é excluído. *Retain* significa que o PV não é excluído; ele apenas entra no estado "*Released*"(Liberado), preservando os dados no NFS. Isso evita a perda acidental de dados de produção. O administrador deve então limpar manualmente o volume e liberar o PV para reutilização.
- **nfs:** Define o *driver* de volume e seus parâmetros. *path* aponta para o diretório exportado no servidor, e *server* aponta para o IP do nosso servidor NFS.

Foi aplicado o manifesto no cluster:

```
1  kubectl apply -f nfs-pv.yaml
```

Este comando envia a definição ao servidor da API do Kubernetes, que cria o objeto e o deixa no estado "*Available*"(Disponível).

² Unidade de medida de armazenamento de dados, criada pela IEC para resolver a confusão com o GigaByte (GB), sendo 1 Gi igual a 2³⁰ bytes (1.073.741.824 bytes), enquanto o GB é decimal (10⁹ bytes ou 1 bilhão de bytes).

2. Criação do PersistentVolumeClaim (PVC):

O PVC (Listing 4.2) atua como uma solicitação de consumo de armazenamento feita pela aplicação. Ele abstrai os detalhes da infraestrutura subjacente, permitindo que o desenvolvedor solicite recursos sem precisar saber o endereço IP do servidor NFS ou o caminho exato do diretório.

```
1  apiVersion: v1
2  kind: PersistentVolumeClaim
3  metadata:
4    name: nfs-pvc-claim-100gi
5  spec:
6    # Define explicitamente que queremos usar o provisionador
   NFS
7    storageClassName: nfs-client
8    accessModes:
9      - ReadWriteMany
10   resources:
11     requests:
12       storage: 100Gi
```

Listing 4.2 – Manifesto nfs-pvc.yaml

A especificação (*spec*) do PVC define os requisitos exatos da carga de trabalho:

- **storageClassName: nfs-client:** Instrui o Kubernetes a utilizar especificamente a classe de armazenamento associada ao *nfs-subdir-external-provisioner*, garantindo que o volume seja criado no backend NFS.
- **accessModes: ReadWriteMany:** Solicita um volume com suporte a montagem simultânea de leitura e escrita por múltiplos nós (RWX), requisito essencial para o teste de concorrência.
- **resources.requests.storage: 100Gi**³: Dimensiona o volume com 100 Gibibytes, conforme estipulado nos cenários de teste.

Neste manifesto, foi especificado o `storageClassName` como `nfs-client`. Com isso, o *controller* de PVs do Kubernetes utilizará a classe de armazenamento associada ao *nfs-subdir-external-provisioner*, criando (ou vinculando) um volume compatível com os requisitos de acesso e tamanho definidos no PVC.

Foi aplicado o manifesto no cluster:

³ Unidade de medida de armazenamento de dados, criada pela IEC para resolver a confusão com o GigaByte (GB), sendo 1 Gi igual a 2^{30} bytes (1.073.741.824 bytes), enquanto o GB é decimal (10^9 bytes ou 1 bilhão de bytes).

```
1 kubectl apply -f nfs-pvc.yaml
```

3. Verificação do Vínculo (Binding):

Após a criação do PVC, o *control plane* do Kubernetes detecta a correspondência e vincula os dois objetos. Podemos verificar isso com o comando `kubectl get`.

```
1 kubectl get pv,pvc
```

O resultado esperado deve mostrar o STATUS de ambos como *Bound* (Vinculado), conforme apresentado no Listing 4.3:

```
1 NAME CAPACITY ACCESS MODES RECLAIM POLICY STATUS CLAIM AGE
   persistentvolume/nfs-pv-share 100Gi RWX Retain Bound
   default/nfs-pvc-claim-100gib 3s
2
3 NAME STATUS VOLUME CAPACITY ACCESS MODES AGE
   persistentvolumeclaim/nfs-pvc-claim-100gib Bound nfs-pv-
   share 100Gi RWX 3s
```

Listing 4.3 – Resultado esperado do comando `kubectl get pv,pvc`

O *output* confirma que o PV `nfs-pv-share` está agora vinculado ao PVC `nfs-pvc-claim-100gi`. O armazenamento está pronto para ser montado por um *pod* que faça referência ao nome deste PVC ⁴.

4.3 Validação da Persistência e Disponibilidade

Após a configuração dos objetos de armazenamento (*PersistentVolume* e *PersistentVolumeClaim*), é mandatório validar se a infraestrutura subjacente está operando conforme o esperado. O objetivo desta etapa é confirmar que um arquivo gravado por um *pod* efêmero é efetivamente persistido no disco físico dedicado (`/dev/sdb`) do servidor NFS e não perdido quando o processo do container termina.

Para este teste, utilizou-se uma imagem leve (`busybox`) para montar o volume e realizar operações de I/O simples.

⁴ Unidade de medida de armazenamento de dados, criada pela IEC para resolver a confusão com o GigaByte (GB), sendo 1 Gi igual a 2³⁰ bytes (1.073.741.824 bytes), enquanto o GB é decimal (10⁹ bytes ou 1 bilhão de bytes).

4.3.1 Definição da Carga de Trabalho (Pod de Teste)

Foi criado um manifesto Kubernetes para instanciar um *pod* que consome o PVC previamente configurado. O manifesto `nfs-pod-teste.yaml` é apresentado no Listing 4.4:

```
1 apiVersion: v1
2 kind: Pod
3 metadata:
4   name: nfs-pod-teste
5 spec:
6   containers:
7     - name: busybox
8       image: busybox
9       # Mantém o container em execução (loop infinito) para
10      permitir acesso via shell
11      command: ["/bin/sh", "-c", "sleep 3600"]
12      volumeMounts:
13        - mountPath: "/data" # Ponto de montagem dentro do
14          container
15          name: nfs-storage
16      volumes:
17        - name: nfs-storage
18          persistentVolumeClaim:
19            claimName: nfs-pvc-claim-100gib # Referência ao PVC
20            criado anteriormente
```

Listing 4.4 – Manifesto do Pod de Teste NFS

Nesta configuração:

- **volumeMounts:** O volume NFS é montado no diretório `/data` dentro do container.
- **claimName:** Vincula este pod especificamente ao PVC de 100Gi⁵ que aponta para o nosso servidor NFS.

O pod foi implantado no cluster com o comando:

```
1 kubectl apply -f nfs-pod-teste.yaml
```

⁵ Unidade de medida de armazenamento de dados, criada pela IEC para resolver a confusão com o GigaByte (GB), sendo 1 Gi igual a 2³⁰ bytes (1.073.741.824 bytes), enquanto o GB é decimal (10⁹ bytes ou 1 bilhão de bytes).

Após a confirmação de que o status do pod mudou para *Running*, procedeu-se com o teste de escrita.

4.3.2 Teste de Escrita e Verificação Física

A validação consiste em escrever um arquivo de dentro do ambiente virtualizado (*Pod*) e verificar sua existência no sistema de arquivos físico do servidor (*Control Plane*).

1. Escrita de dados (Dentro do *Pod*):

Utilizando o comando `kubectl exec`, acessa-se o shell do container e cria-se um arquivo de texto contendo um *timestamp*.

```
1
2 #Acessa o terminal interativo do Pod
3 kubectl exec -it nfs-pod-teste -- sh
4
5 #Navega at o diretório montado via NFS
6 echo "Teste real do NFS - $(date)" > teste-real.txt
7
8 #Confirma a criação no contexto do container
9 ls -l teste-real.txt
10
11 #Encerra a sessão
12 exit
```

Listing 4.5 – Operação de escrita dentro do Pod

2. Verificação de Persistência (No Servidor Físico):

Para confirmar que os dados atravessaram a rede e foram gravados no disco rígido secundário do *Control Plane*, foi acessado o diretório exportado pelo servidor NFS (`/mnt/dados/kubedata`).

```
1 #Comando executado no terminal do Control Plane (Servidor NFS)
2 cat /mnt/dados/kubedata/teste-real.txt
```

Listing 4.6 – Verificação do arquivo no disco do servidor

Isso comprova o ciclo completo de persistência:

Aplicação (Pod) → Kubelet (Worker Node) → Rede → NFS Server (Control Plane) → Disco Físico (/dev/sdb).

4.4 Cenário 2: Implantação de GlusterFS

O GlusterFS é um sistema de arquivos distribuído e escalável que agrega recursos de armazenamento em disco de vários servidores em um único espaço de nomes global. Diferente do NFS tradicional, que apresenta um ponto único de falha, o GlusterFS permite a replicação síncrona de dados entre nós, garantindo alta disponibilidade.

Neste cenário, foi configurada uma arquitetura hiperconvergente, onde os nós de trabalho (*worker nodes*) atuam simultaneamente como servidores de armazenamento e clientes do cluster Kubernetes. A topologia implementada, detalhando a interação entre os componentes de armazenamento e a camada de orquestração, está ilustrada na Figura 10.

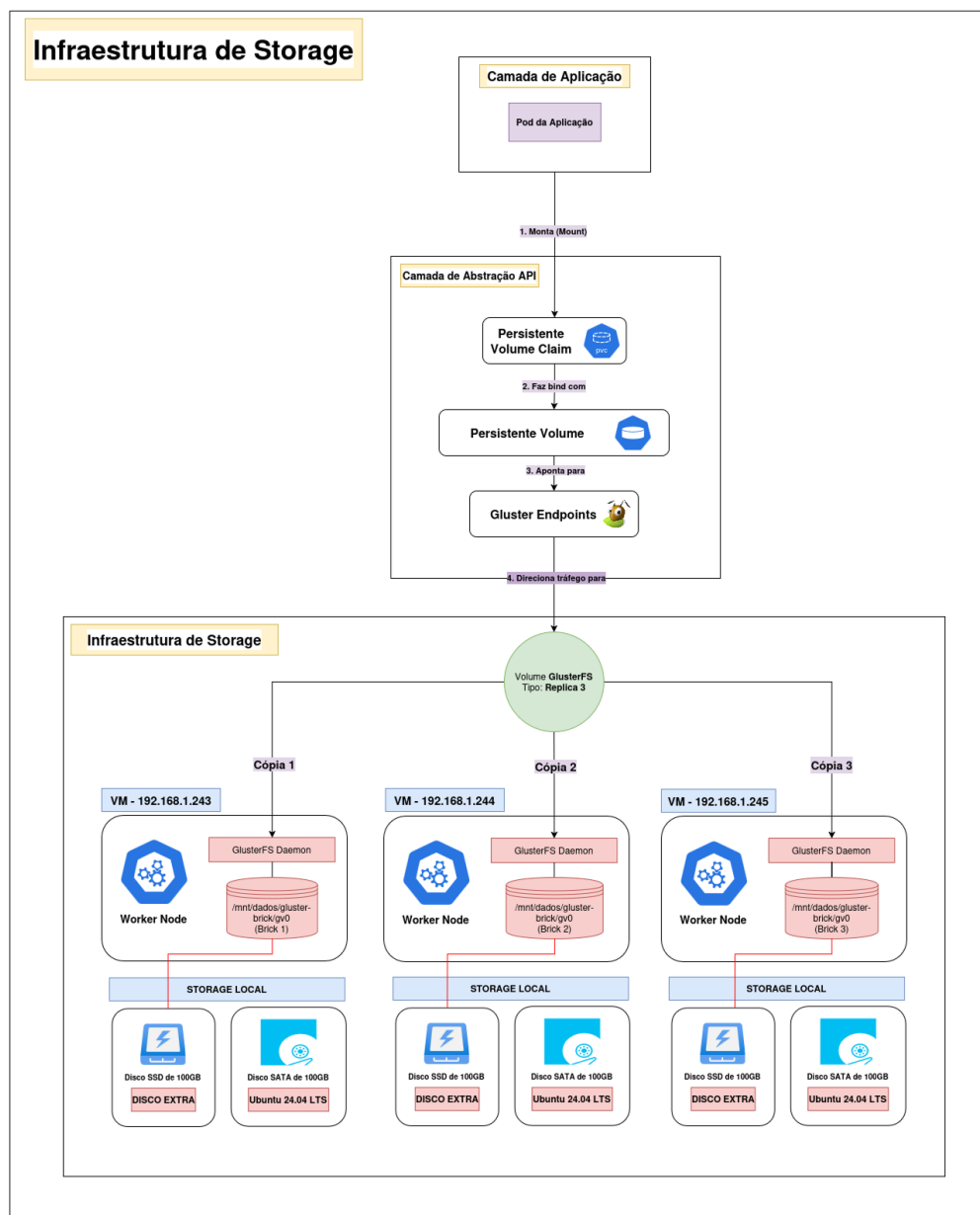


Figura 10 – Arquitetura do Cenário 02: Cluster Kubernetes com GlusterFS Hiperconvergente

Fonte: Autoria própria (2025).

4.4.1 Instalação e Preparação dos Nós (Worker Nodes)

Os procedimentos a seguir foram executados em todos os três nós de trabalho (192.168.1.243, 192.168.1.244 e 192.168.1.245) para preparar a infraestrutura física e lógica do sistema de arquivos.

1. Instalação e Inicialização do Serviço:

O pacote `glusterfs-server` fornece os binários necessários para a criação do cluster de armazenamento (servidor) e a montagem dos volumes (cliente).

```
1 sudo apt update
2 sudo apt install glusterfs-server -y
3
```

Após a instalação, o serviço `glusterd` (Gluster Daemon) foi habilitado para iniciar no *boot* e executado imediatamente. Este serviço é responsável pela gestão da topologia do cluster e sincronização de configurações (Gluster Community, 2025).

```
1 sudo systemctl enable --now glusterd
2 sudo systemctl status glusterd
3
```

2. Preparação do *Brick* e Ponto de Montagem:

No GlusterFS, um *Brick* é a unidade fundamental de armazenamento. Considerando que o disco secundário (`/dev/sdb`) já foi formatado e montado em `/mnt/dados` em cada nó, cria-se o diretório do brick dentro deste ponto de montagem:

```
1 sudo mkdir -p /mnt/dados/gluster-brick/gv0
2
```

Adicionalmente, criou-se o diretório `/mnt/k8s-data`. Este diretório servirá como ponto de montagem local para o volume distribuído, permitindo que o Kubernetes acesse o armazenamento via driver `hostPath`.

```
1 sudo mkdir -p /mnt/k8s-data
2
```

4.4.2 Configuração do Trusted Storage Pool e Volume

A configuração lógica do cluster foi centralizada no *Worker Node 01* (192.168.1.243), propagando-se automaticamente para os demais pares.

1. Estabelecimento de Confiança (*Peer Probe*):

Para formar o *Trusted Storage Pool*, os nós devem ser explicitamente registrados. O comando `peer probe` envia uma solicitação de *handshake* para os demais nós.

```
1 sudo gluster peer probe 192.168.1.244
2 sudo gluster peer probe 192.168.1.245
3
```

A validação da conectividade foi feita verificando se o status dos pares constava como *"Connected"*.

```
1 sudo gluster peer status
2
```

2. Criação e Inicialização do Volume Replicado:

O volume distribuído `k8s-volume` foi criado utilizando a topologia *Replica 3*. Esta configuração assegura que cada arquivo gravado possua cópias idênticas em três nós distintos, armazenadas no disco dedicado (`/mnt/dados`).

```
1 # Cria o do volume apontando para os bricks no disco
   secundario
2 sudo gluster volume create k8s-volume replica 3 \
3   192.168.1.243:/mnt/dados/gluster-brick/gv0 \
4   192.168.1.244:/mnt/dados/gluster-brick/gv0 \
5   192.168.1.245:/mnt/dados/gluster-brick/gv0 \
6   force
7
8 sudo gluster volume start k8s-volume
9
```

Nota: O parâmetro `force` é utilizado para suprimir avisos de segurança padrão durante a criação do volume, garantindo a execução em ambiente de laboratório.

3. Montagem do Volume no Sistema Operacional:

Para expor o armazenamento distribuído ao Kubernetes, o volume GlusterFS foi montado localmente em *todos os worker nodes*. Este passo abstrai a complexidade da rede para o orquestrador.

```
1 # Executado em todos os workers
2 sudo mount -t glusterfs localhost:/k8s-volume /mnt/k8s-data
3
```

O uso de `localhost` é possível devido à arquitetura hiperconvergente, onde o cliente e o servidor residem na mesma máquina. A verificação foi feita com:

```
1 df -h | grep k8s-data
2
```

4.4.3 Integração com Kubernetes (PV e PVC)

Com o sistema de arquivos montado no nível do SO em `/mnt/k8s-data` em todos os nós, configurou-se a persistência no Kubernetes utilizando a estratégia `hostPath`. Embora o `hostPath` normalmente aponte para um diretório local isolado, neste cenário ele aponta para o ponto de montagem do GlusterFS, tornando o dado efetivamente distribuído.

1. Criação do PersistentVolume (PV):

O manifesto `gluster-hostpath-pv.yaml` define o recurso de armazenamento. Note que o `accessModes` é definido como *ReadWriteMany* (RWX), suportado nativamente pelo GlusterFS.

```
1  apiVersion: v1
2  kind: PersistentVolume
3  metadata:
4    name: gluster-pv-local
5  spec:
6    capacity:
7      storage: 100Gi
8    volumeMode: Filesystem
9    accessModes:
10     - ReadWriteMany
11    persistentVolumeReclaimPolicy: Retain
12    storageClassName: manual
13    hostPath:
14      path: /mnt/k8s-data
15      type: Directory
16
```

Aplicação do manifesto:

```
1  kubectl apply -f gluster-hostpath-pv.yaml
2
```

2. Criação do PersistentVolumeClaim (PVC):

A solicitação de armazenamento foi definida no arquivo `gluster-hostpath-pvc.yaml`, vinculando-se ao PV criado anteriormente através da `storageClassName: manual`.

```
1  apiVersion: v1
2  kind: PersistentVolumeClaim
3  metadata:
4    name: gluster-pvc-claim
```

```
5 spec:
6   accessModes:
7     - ReadWriteMany
8   storageClassName: manual
9   resources:
10    requests:
11      storage: 100Gi
12
```

Aplicação do manifesto:

```
1 kubectl apply -f gluster-hostpath-pvc.yaml
2
```

4.4.4 Validação da Persistência e Replicação

Para validar a funcionalidade *ReadWriteMany* e a replicação dos dados, implantou-se um *Pod* de teste que escreve periodicamente no volume.

1. Implantação do Pod de Teste:

O arquivo `gluster-test-pod.yaml` define um container *Busybox* que grava a data atual em um arquivo de log dentro do volume compartilhado.

```
1 apiVersion: v1
2 kind: Pod
3 metadata:
4   name: gluster-test-pod
5 spec:
6   containers:
7     - name: test-container
8       image: busybox
9       command: ["/bin/sh", "-c", "while true; do date >> /
10         data/gluster-check.txt; sleep 5; done"]
11       volumeMounts:
12         - name: gluster-storage
13           mountPath: "/data"
14   volumes:
15     - name: gluster-storage
16       persistentVolumeClaim:
17         claimName: gluster-pvc-claim
```



```
1 kubectl apply -f gluster-test-pod.yaml
2 # Monitoramento da inicializa o do Pod
3 kubectl get pod gluster-test-pod -w
4
```

2. Verificação da Replicação de Dados:

O teste definitivo de um sistema de arquivos distribuído consiste em verificar a disponibilidade dos dados em um nó diferente daquele onde o *Pod* está sendo executado. Acessou-se o terminal de um *worker node* distinto e verificou-se o conteúdo do arquivo gerado:

```
1 # Executado em um n diferente daquele que hospeda o Pod
2 cat /mnt/k8s-data/gluster-check.txt
3
```

A visualização dos dados atualizados confirmou que o `hostPath` estava lendo do volume GlusterFS replicado, validando a arquitetura de armazenamento distribuído em alta disponibilidade.

4.5 Cenário 3: Implantação de CephFS via Rook

O Ceph é uma plataforma de armazenamento distribuído unificada, projetada para oferecer alto desempenho, confiabilidade e escalabilidade em escala de exabytes. Para integrar essa solução ao Kubernetes, utilizou-se o *Rook*, um orquestrador de armazenamento nativo da nuvem. O Rook atua seguindo o padrão de *Operator* do Kubernetes, automatizando tarefas complexas como implantação, *bootstrapping*, configuração, provisionamento, escalonamento e recuperação de desastres do cluster Ceph (Rook Project, 2025).

Uma distinção fundamental deste cenário em relação aos anteriores (NFS e GlusterFS) reside na camada de persistência física. O Ceph, quando gerenciado pelo Rook, exige acesso exclusivo a dispositivos de armazenamento em bloco brutos (*raw block devices*), sem partições ou sistemas de arquivos pré-existent. Isso permite que os *Object Storage Daemons* (OSDs) gerenciem diretamente a alocação de dados no disco, otimizando a performance de I/O.

A arquitetura implementada, detalhando a interação entre o Operador Rook, os componentes do Ceph e a camada de abstração do Kubernetes, está ilustrada na Figura 11.

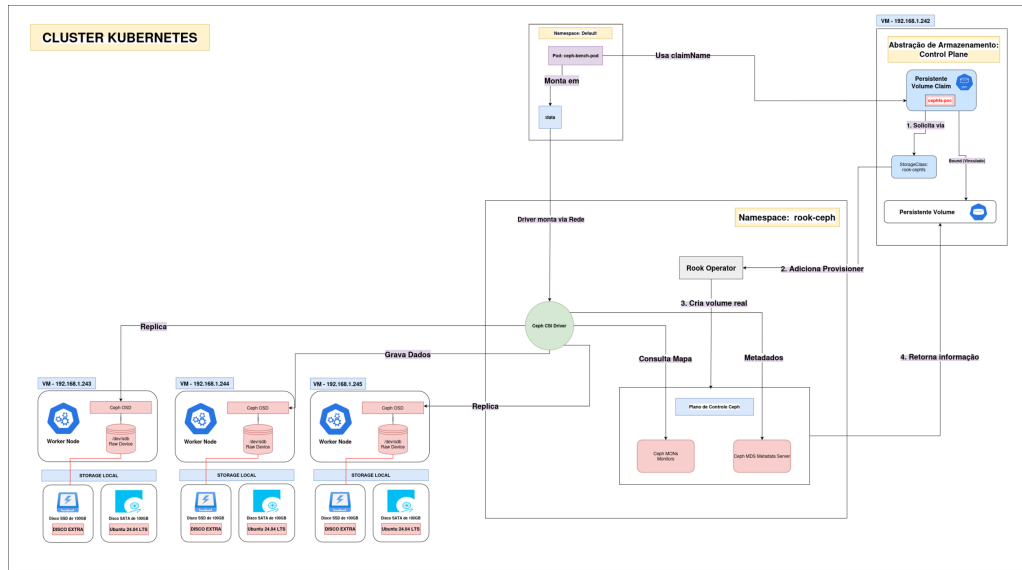


Figura 11 – Arquitetura do Cenário 03: Cluster Kubernetes com CephFS gerenciado pelo Rook

Fonte: Autoria própria (2025).

4.5.1 Preparação dos Dispositivos de Armazenamento

A etapa crítica para o funcionamento do Ceph é garantir que os discos destinados ao armazenamento não possuam partições ou sistemas de arquivos pré-existent. O procedimento a seguir foi executado em todos os três *worker nodes* para limpar o disco secundário (`/dev/sdb`).

```

1 # Remove todas as assinaturas de sistema de arquivos ou
   particoes
2 sudo wipefs -a /dev/sdb
3
4 # Verificacao (o comando nao deve retornar output para sdb)
5 lsblk -f /dev/sdb

```

Listing 4.7 – Limpeza dos discos para uso do Ceph

4.5.2 Implantação do Operador Rook

O Operador do Rook é o componente que estende a API do Kubernetes, adicionando CRDs (*Custom Resource Definitions*) que permitem gerenciar o cluster Ceph como objetos nativos do Kubernetes.

1. **Obtenção dos Manifestos:** Utilizou-se a versão estável v1.14.0 do projeto Rook.

```
1 git clone --single-branch --branch v1.14.0 https://github.  
  com/rook/rook.git  
2 cd rook/deploy/examples  
3
```

- 2. Instalação dos Recursos:** Foram aplicados os CRDs, as configurações comuns de namespace e políticas de segurança (RBAC), e finalmente o manifesto do Operador.

```
1 kubectl apply -f crds.yaml  
2 kubectl apply -f common.yaml  
3 kubectl apply -f operator.yaml  
4
```

A verificação da disponibilidade do Operador foi realizada monitorando o namespace rook-ceph:

```
1 kubectl get pods -n rook-ceph -w  
2
```

4.5.3 Definição e Criação do Cluster Ceph

Com o operador ativo, definiu-se a topologia do cluster de armazenamento. O arquivo `cluster.yaml` (Listing 4.8) foi editado para especificar explicitamente quais nós e dispositivos seriam utilizados, garantindo que o Ceph consumisse apenas o disco `sdb` preparado na etapa inicial.

```
1 storage:  
2   useAllNodes: false  
3   useAllDisks: false  
4   nodes:  
5     - name: "worker-node-01"  
6       devices:  
7         - name: "sdb"  
8     - name: "worker-node-02"  
9       devices:  
10        - name: "sdb"  
11    - name: "worker-node-03"  
12      devices:  
13        - name: "sdb"
```

Listing 4.8 – Configuração de armazenamento no `cluster.yaml`

Após a aplicação deste manifesto (`kubectl apply -f cluster.yaml`), o Rook iniciou a preparação dos OSDs e dos monitores (MONs) do Ceph.

4.5.4 Configuração do Sistema de Arquivos (CephFS)

Para habilitar o armazenamento de arquivos compartilhados (necessário para RWX), foi instanciado o CephFS. Esta ação dispara a criação dos *Metadata Servers* (MDS), responsáveis por gerenciar a estrutura de diretórios e permissões dos arquivos.

```
1 kubectl apply -f filesystem.yaml
2 # Monitoramento da inicializa o dos MDS (Metadata Servers)
3 kubectl get pods -n rook-ceph -l app=rook-ceph-mds
```

4.5.5 Configuração da Classe de Armazenamento (StorageClass)

Para permitir o provisionamento dinâmico de volumes, configurou-se uma StorageClass associada ao driver CSI (*Container Storage Interface*) do CephFS.

```
1 cd ../csi/cephfs/
2 kubectl apply -f storageclass.yaml
```

Esta configuração abstrai a complexidade do Ceph, permitindo que o Kubernetes solicite volumes automaticamente quando um PVC é criado.

4.5.6 Provisionamento de Volume (PVC) e Pré-requisitos do Cliente

Diferentemente dos cenários de NFS e GlusterFS, onde o PV foi criado manualmente, no cenário com Rook utilizou-se o provisionamento dinâmico. Criou-se apenas a Solicitação de Volume Persistente (PVC), conforme Listing 4.9.

```
1 apiVersion: v1
2 kind: PersistentVolumeClaim
3 metadata:
4   name: cephfs-pvc
5 spec:
6   storageClassName: rook-cephfs
7   accessModes:
8     - ReadWriteMany
9   resources:
10     requests:
11       storage: 50Gi
```

Listing 4.9 – Manifesto ceph-pvc.yaml

Aplicação do manifesto:

```
1 kubectl apply -f ceph-pvc.yaml
```

Instalação de Dependências nos Clientes: Para que os nós do Kubernetes consigam montar os volumes CephFS provisionados, é necessário que o kernel do sistema operacional possua os módulos e utilitários clientes. O comando a seguir foi executado em *todos os worker nodes*:

```
1 sudo apt update && sudo apt install ceph-common -y
2 # Carregamento manual do módulo do kernel para validar o
3 sudo modprobe ceph
```

4.5.7 Validação de Acesso Concorrente (RWX)

Para validar a capacidade *ReadWriteMany* do CephFS, realizou-se um teste prático com dois *Pods* acessando o mesmo *PVC* simultaneamente.

1. **Pod de Escrita:** O primeiro Pod monta o volume e grava um arquivo de teste (Listing 4.10).

```
1 apiVersion: v1
2 kind: Pod
3 metadata:
4   name: ceph-pod-teste
5 spec:
6   containers:
7     - name: busybox
8       image: busybox
9       command: ["/bin/sh", "-c", "sleep 3600"]
10      volumeMounts:
11        - mountPath: "/data"
12          name: ceph-storage
13    volumes:
14      - name: ceph-storage
15        persistentVolumeClaim:
16          claimName: cephfs-pvc
17
```

Listing 4.10 – Manifesto ceph-pod.yaml

Após a aplicação e inicialização do pod, executou-se a escrita do dado:

```
1 kubectl exec -it ceph-pod-teste -- sh -c "echo 'Dados  
persistidos no CephFS' > /data/teste-rwx.txt"
```

2. **Pod de Leitura:** Um segundo Pod, idêntico ao primeiro mas com nome distinto (ceph-pod-teste-2), foi criado utilizando o **mesmo PVC**. O objetivo foi verificar se o dado escrito pelo primeiro Pod estava imediatamente disponível para o segundo, validando o sistema de arquivos distribuído.

```
1 # Leitura do arquivo a partir do segundo Pod  
2 kubectl exec -it ceph-pod-teste-2 -- cat /data/teste-rwx.txt  
3
```

A confirmação da leitura do arquivo `teste-rwx.txt` pelo segundo Pod validou com sucesso a implementação do CephFS com suporte a acesso concorrente de escrita e leitura (RWX).

4.6 Protocolo de Testes de Desempenho

Para garantir a comparabilidade dos resultados entre as três arquiteturas de armazenamento (NFS, GlusterFS e CephFS), foi definido um protocolo de testes padronizado. O objetivo central consistiu em mensurar a latência de escrita (*latency*) e a vazão efetiva (*throughput*) de dados persistidos no disco, isolando variáveis como cache de memória e processamento de aplicação.

4.6.1 Carga de Trabalho (Workload)

A carga de trabalho foi definida como operações de escrita sequencial de arquivos binários. Foram gerados quatro artefatos de teste com tamanhos progressivos: 1 GB, 3 GB, 5 GB e 7 GB. A escolha destes tamanhos visou avaliar o comportamento do sistema tanto em cenários onde os dados cabem inteiramente na memória RAM (cache do Linux) quanto em cenários de saturação, onde a escrita física em disco se torna mandatória.

A geração dos arquivos foi realizada utilizando o utilitário `dd`, lendo a partir de `/dev/zero` para minimizar o custo de CPU na geração dos dados, conforme detalhado na Listing 4.11.

```
1 dd if=/dev/zero of=/tmp/arquivo_1GB.img bs=1G count=1 status=  
progress
```

```
2 dd if=/dev/zero of=/tmp/arquivo_3GB.img bs=1G count=3 status=  
   progress  
3 dd if=/dev/zero of=/tmp/arquivo_5GB.img bs=1G count=5 status=  
   progress  
4 dd if=/dev/zero of=/tmp/arquivo_7GB.img bs=1G count=7 status=  
   progress
```

Listing 4.11 – Comandos para geração da carga de trabalho

4.6.2 Automação e Coleta de Métricas

A execução dos testes foi automatizada por meio de um *shell script* desenvolvido para iterar 50 vezes a operação de cópia (cp) para cada tamanho de arquivo. Para garantir a precisão das métricas de persistência, o comando de cópia foi seguido imediatamente pela instrução `sync`. Esta instrução força o sistema operacional a descarregar (*flush*) todos os *buffers* de escrita da memória volátil para o meio de armazenamento persistente antes de considerar a operação concluída.

O tempo de execução foi mensurado através do utilitário `time`, coletando-se três métricas principais:

- **Real:** Tempo total de relógio decorrido (latência percebida pelo usuário).
- **User:** Tempo de CPU gasto em modo usuário.
- **Sys:** Tempo de CPU gasto em modo kernel (chamadas de sistema).

A Listagem 4.12 apresenta a estrutura lógica do script utilizado em todos os cenários.

```
1 #!/bin/bash  
2  
3 # Configuracoes do Experimento  
4 LOOPS=50  
5 SOURCE_DIR="/tmp"  
6 DEST_DIR="/mnt" # Ponto de montagem no Worker  
7  
8 # Vetor com os nomes dos arquivos gerados previamente  
9 FILES=("arquivo_1GB.img" "arquivo_3GB.img" "arquivo_5GB.img" "  
   arquivo_7GB.img")  
10  
11 echo "--- Iniciando Bateria de Testes de Persistencia ---"  
12 echo "Destino NFS: $DEST_DIR | Repeti es: $LOOPS"
```

```
13
14 # Validacao do ambiente
15 if [ ! -d "$DEST_DIR" ]; then
16     echo "ERRO CRITICO: Diretorio $DEST_DIR nao encontrado!
17     Verifique a montagem."
18     exit 1
19 fi
20 for FILE in "${FILES[@]"; do
21     SOURCE_PATH="$SOURCE_DIR/$FILE"
22     OUTPUT_CSV="resultado_nfs_${FILE}.csv"
23
24     echo "===== "
25     echo "Iniciando bateria para: $FILE"
26
27     if [ ! -f "$SOURCE_PATH" ]; then
28         echo "AVISO: Arquivo fonte $SOURCE_PATH inexistente.
29         Pulando..."
30         continue
31     fi
32
33     # Cabecalho do arquivo CSV
34     echo "Real(s);User(s);Sys(s);CPU_Usage" > $OUTPUT_CSV
35
36     for i in $(seq 1 $LOOPS); do
37         echo "    -> [ $FILE ] Execucao $i de $LOOPS..."
38
39         # Remove o arquivo anterior para forcar nova
40         transferencia
41         rm -f "$DEST_DIR/$FILE"
42
43         # Executa copia + sync e coleta metricas de tempo
44         /usr/bin/time -f "%e;%U;%S;%P" sh -c "cp $SOURCE_PATH
45         $DEST_DIR/ && sync" 2>> $OUTPUT_CSV
46     done
47
48     echo "Finalizado para $FILE. Dados salvos em: $OUTPUT_CSV"
49 done
50
51 echo "===== "
52 echo "Experimento concluido com sucesso."
```


Listing 4.12 – Estrutura do script de benchmark padronizado

4.6.3 Estratégias de Execução por Cenário

Embora o protocolo lógico (*script* e carga de trabalho) seja idêntico para todos os experimentos, a estratégia de execução física variou conforme a arquitetura de armazenamento para garantir a fidelidade da medição:

- **Cenários Baseados em Sistema Operacional (NFS e GlusterFS):** Para estas tecnologias, o volume distribuído foi montado diretamente no sistema de arquivos do *Worker Node* (ex: em `/mnt`). O script foi executado no terminal do *host*, avaliando a performance da montagem via kernel/FUSE.
- **Cenário Nativo de Contêiner (CephFS):** Para o CephFS, gerenciado pelo Rook, optou-se pela execução *intra-pod*. Um *Pod* de ferramentas ("*toolbox*") foi implantado no cluster montando o PVC do CephFS. O *script* foi executado de dentro deste contêiner, validando a pilha completa de I/O do Kubernetes (CSI Driver).

A Figura 12 ilustra o fluxo metodológico de execução dos testes de desempenho aplicado no estudo.

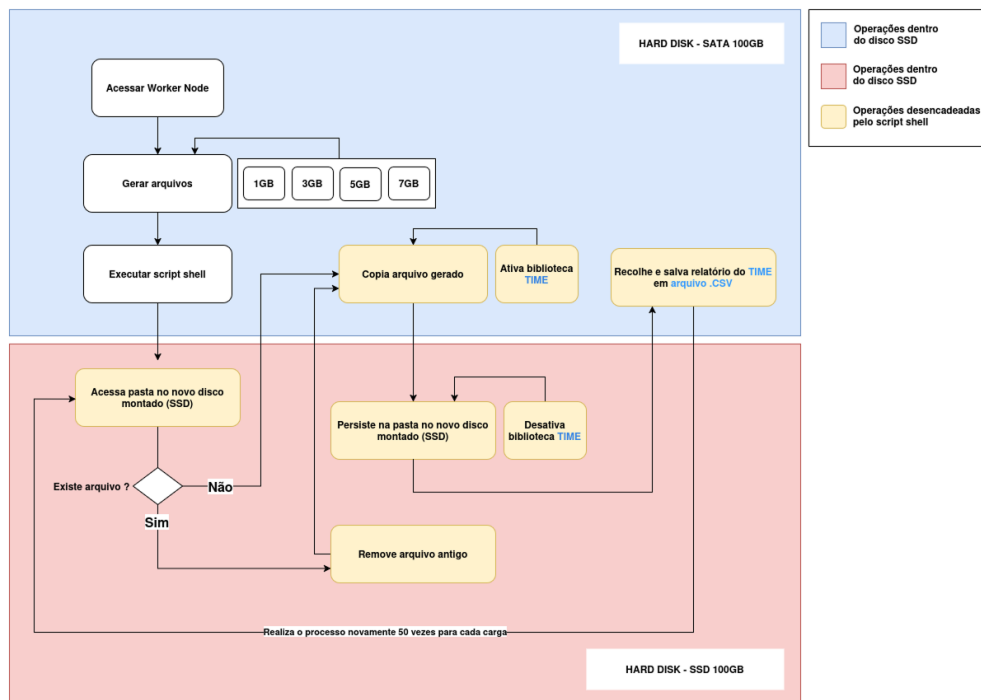


Figura 12 – Fluxo esquemático de execução dos testes de desempenho

CAPÍTULO 5

RESULTADOS

Este capítulo apresenta e analisa os dados quantitativos coletados durante a execução dos testes de estresse, conforme detalhado no capítulo anterior. Os resultados são comparados em três eixos principais: latência de escrita, desempenho de vazão (*Throughput*) e o *overhead* de recursos (CPU) imposto por cada solução de armazenamento.

Todos os arquivos dos resultados gerados estão disponíveis no GitHub¹.

5.1 NFS (Network File System)

Os testes de desempenho realizados sobre a infraestrutura NFS, operando em modelo centralizado com disco dedicado no *Control Plane*, geraram os resultados de latência e vazão apresentados na Tabela 1. O conjunto de dados inclui um cenário estendido com arquivos de 10GiB para validação de comportamento sob estresse intenso.

¹ Disponível em: <<https://github.com/pereiraR3/TCC-NFS-GLUSTER-CEPH>>

Tabela 1 – Métricas de Desempenho de Escrita - NFS

Arquivo	Tempo Real (s)	Tempo User (s)	Tempo Sys (s)	Vazão Média (MiB/s)
1GiB	2.22	0.00	1.26	461.93
3GiB	5.58	0.01	3.84	550.95
5GiB	8.12	0.02	6.38	630.26
7GiB	35.69	0.03	11.66	200.83
10GiB	38.60	0.04	16.03	265.26

Fonte: Autoria própria (2025).

5.1.1 Análise de Desempenho

A análise dos dados evidencia uma escalabilidade linear positiva da vazão (*throughput*) nos cenários iniciais, culminando em um pico de desempenho de 630,26 MiB/s com o arquivo de 5GiB. O tempo de sistema (*Sys Time*) acompanhou esse crescimento de forma proporcional, indicando que, até este ponto, o *overhead* de protocolo e chamadas de sistema não representou um gargalo.

No entanto, identifica-se uma ruptura crítica de desempenho a partir do teste de 7GiB. A vazão sofreu uma queda abrupta para 200,83 MiB/s, comportamento que foi corroborado pelo teste suplementar de 10GiB (265,26 MiB/s). Este novo patamar de performance, significativamente inferior aos cenários anteriores, confirma a hipótese de saturação do *cache de página* (buffer de memória RAM) no servidor NFS.

Enquanto os arquivos de até 5GiB foram acomodados quase inteiramente na memória volátil antes da descarga (*flush*) para o disco, os arquivos maiores excederam a capacidade disponível de *buffer*. Isso forçou o sistema a transicionar para um regime de escrita síncrona direta no disco físico (`/dev/sdb`), expondo a latência real do hardware de armazenamento e limitando a velocidade à capacidade física do dispositivo.

5.1.2 Consumo de Recursos

O monitoramento de recursos revelou uma correlação direta entre a vazão obtida e a utilização de CPU no *Worker Node*. Nos cenários de alta vazão (até 5GiB), o uso de CPU atingiu o pico de 78%, refletindo o trabalho intenso de empacotamento e envio de dados pela rede.

Em contrapartida, nos cenários de saturação (7GiB e 10GiB), a utilização de CPU caiu para aproximadamente 32%. Essa redução não indica eficiência, mas sim ociosidade forçada (*iowait*): o processador do cliente permaneceu a maior parte do tempo aguardando a confirmação de escrita (I/O) por parte do servidor NFS, que se tornou o gargalo do sistema ao operar no limite físico do disco.

5.2 GlusterFS

Para o GlusterFS, configurado em topologia hiperconvergente com replicação tripla (Replica 3), os dados de persistência obtidos estão sumarizados na Tabela 2. É fundamental notar que, devido ao fator de replicação, cada operação de escrita lógica solicitada pelo cliente resultou em três operações de escrita física distribuídas simultaneamente pela rede.

Tabela 2 – Métricas de Desempenho de Escrita - GlusterFS

Arquivo	Tempo Real (s)	Tempo User (s)	Tempo Sys (s)	Vazão Média (MiB/s)
1GiB	6.48	0.02	1.56	158.10
3GiB	19.17	0.08	4.91	160.28
5GiB	32.68	0.14	8.41	156.65
7GiB	62.79	0.21	16.28	114.16

Fonte: Autoria própria (2025).

5.2.1 Análise de Desempenho

A análise dos dados revela uma vazão relativamente estável entre 1GiB e 5GiB (na faixa de 156 a 160 MiB/s), refletindo o custo fixo da replicação tripla do GlusterFS e o tráfego de rede necessário para manter as cópias sincronizadas. O tempo de sistema (*Sys Time*) apresentou crescimento linear, indicando que o *overhead* de chamadas de sistema no cliente é proporcional ao volume de dados.

O ponto de inflexão ocorre no teste de 7GiB, onde houve uma degradação de desempenho, com a vazão caindo para 114,16 MiB/s. Esse comportamento sugere saturação combinada de rede e disco local, agravada pelo *overhead* da replicação síncrona e do algoritmo de distribuição (DHT), resultando em maior latência efetiva para gravações maiores.

5.2.2 Consumo de Recursos

Diferente da implementação de kernel do NFS, o GlusterFS opera majoritariamente em espaço de usuário. Isso resultou em um *overhead* computacional distinto nos *Worker Nodes*. O processo `glusterfsd` consumiu ciclos de CPU adicionais para o cálculo de *hashes* (necessários para o algoritmo de distribuição elástica - DHT) e para a gestão da replicação síncrona. Esse cenário evidencia o impacto da arquitetura hiperconvergente, onde a camada de computação compete diretamente com a camada de armazenamento pelos mesmos recursos de processamento.

5.3 CephFS

O CephFS, orquestrado via Rook e acessado através do driver CSI (*Container Storage Interface*), apresentou os resultados detalhados na Tabela 3. Os testes ocorreram dentro do ambiente de *Pod*, exercitando a pilha completa de virtualização de armazenamento, desde a aplicação até os OSDs.

Tabela 3 – Métricas de Desempenho de Escrita - CephFS

Arquivo	Tempo Real (s)	Tempo User (s)	Tempo Sys (s)	Vazão Média (MiB/s)
1GiB	6.74	0.00	0.97	151.95
3GiB	19.31	0.00	4.05	159.13
5GiB	31.38	0.01	8.93	163.16
7GiB	46.71	0.01	14.77	153.46

Fonte: Autoria própria (2025).

5.3.1 Análise de Desempenho

Diferente das soluções anteriores (NFS e GlusterFS), que apresentaram degradação de desempenho com o aumento da carga (arquivo de 7GiB), o CephFS demonstrou uma estabilidade notável na vazão de dados, mantendo-se consistentemente na faixa de 150 a 163 MiB/s para todos os tamanhos de arquivo.

Embora a latência absoluta ("Tempo Real") seja superior à do NFS devido à complexidade do algoritmo CRUSH (que calcula deterministicamente a localização dos dados) e à replicação síncrona distribuída, a previsibilidade do desempenho destaca o CephFS como a solução mais robusta para cargas de trabalho sustentadas e intensivas.

5.3.2 Consumo de Recursos

Em contrapartida à estabilidade, o CephFS apresentou o *overhead* de CPU mais elevado entre as três soluções. A arquitetura baseada em microsserviços do Rook, que exige a execução de múltiplos *Pods* de suporte (OSDs, MONs, MGRs) juntamente com os contêineres *sidecar* do driver CSI em cada nó, gera uma carga basal constante no sistema. Durante as operações de escrita, o uso de CPU aumentou linearmente conforme o tempo de sistema ("Tempo Sys") na Tabela 3, refletindo o custo computacional da fragmentação dos arquivos em objetos e do cálculo de *placement groups* em tempo real.

5.4 Análise Comparativa dos Cenários

Para consolidar a análise comparativa entre as três soluções estudadas (NFS, GlusterFS e CephFS), a vazão média (*Throughput*) foi calculada para cada cenário de teste.

Esta métrica representa a velocidade efetiva de persistência dos dados no subsistema de armazenamento.

O cálculo, que converte o volume de dados processado pelo tempo decorrido, é definido pela Equação 5.1:

$$V = \frac{S_{arg} \times 1024}{T_{real}} \quad (5.1)$$

Onde:

- V : Vazão média calculada em Mebibytes por segundo (MiB/s);
- S_{arg} : Tamanho do arquivo de teste em Gibibytes (GiB);
- 1024: Fator de conversão de unidades (GiB para MiB);
- T_{real} : Tempo real (*wall-clock time*) decorrido para a conclusão da operação de escrita e sincronização (*sync*), medido em segundos.

A Figura 13 sintetiza visualmente os resultados obtidos, evidenciando a queda de desempenho do NFS em arquivos grandes e a estabilidade das soluções distribuídas (GlusterFS e CephFS).

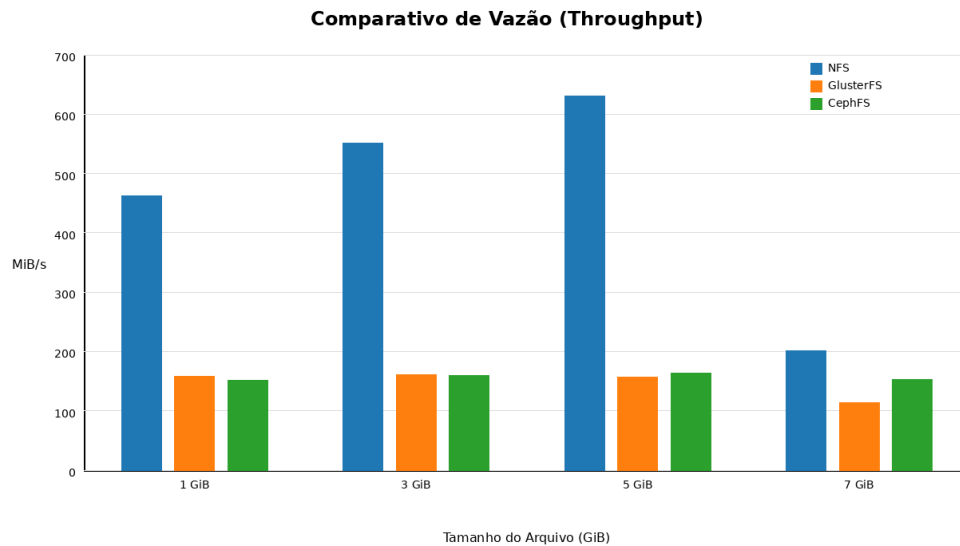


Figura 13 – Comparativo de Vazão Média de Escrita entre as soluções (Maior é melhor)

CAPÍTULO 6

CONCLUSÃO

O presente trabalho teve como objetivo analisar e comparar o desempenho de três soluções distintas de armazenamento persistente (NFS, GlusterFS e CephFS) aplicadas a um cluster Kubernetes. A orquestração de contêineres, embora resolva problemas complexos de implantação de aplicações, introduz desafios significativos na camada de persistência de dados, exigindo que a infraestrutura de armazenamento acompanhe a natureza dinâmica e distribuída do ambiente.

Os experimentos realizados permitiram observar claramente os *trade-offs* entre simplicidade, desempenho e escalabilidade:

- **NFS:** Demonstrou ser a solução de implementação mais simples e com a menor latência para cargas de trabalho moderadas (arquivos de até 5GiB), beneficiando-se da ausência de *overhead* de replicação de rede. Contudo, os testes de estresse com arquivos de 7GiB expuseram sua fragilidade arquitetural: a saturação dos *buffers* de memória resultou em uma degradação abrupta de desempenho (queda de aproximadamente 68% na vazão). Além disso, por representar um Ponto Único de Falha (SPOF) na configuração centralizada, sua aplicação em ambientes críticos de produção é limitada.
- **GlusterFS:** Apresentou-se como uma solução intermediária viável para transformar nós de computação em armazenamento distribuído (hiperconvergência). Embora

tenha garantido a disponibilidade dos dados através da replicação, pagou-se um preço alto em latência e uso de CPU nos *Worker Nodes*, devido ao processamento em espaço de usuário (FUSE) e à replicação síncrona dos dados.

- **CephFS:** Embora possua a curva de aprendizado e complexidade de implantação mais elevadas — mitigadas parcialmente pelo uso do Operador Rook —, o CephFS provou ser a solução mais robusta. Os resultados demonstraram uma estabilidade de vazão superior em todos os cenários de teste, não sofrendo as quedas de desempenho observadas nas outras soluções quando submetido a grandes volumes de dados. A complexidade do algoritmo CRUSH e a arquitetura de microserviços (MONs, OSDs, MDSs) geram um consumo basal de recursos maior, mas entregam em troca previsibilidade e escalabilidade linear.

Conclui-se que não existe uma solução "bala de prata". Para ambientes de desenvolvimento ou cargas de leitura intensiva e baixo volume, o NFS continua eficiente. Para clusters pequenos que exigem HA sem a complexidade de gestão de discos brutos, o GlusterFS atende bem. Porém, para ambientes de produção Kubernetes que exigem consistência, resiliência e desempenho sustentado sob carga pesada, o CephFS consolidou-se como a escolha superior neste estudo.

Adicionalmente, o desenvolvimento desta pesquisa permitiu avaliar a curva de aprendizado necessária para a transição de orquestradores mais simplificados, como Docker Swarm e Docker Compose, para o ecossistema Kubernetes. Observou-se que, embora o Kubernetes apresente uma barreira de entrada inicial elevada devido à sua robustez e complexidade arquitetural, a assimilação das ferramentas, configurações e técnicas de implantação de sistemas de arquivos distribuídos é viável e progressiva, demonstrando que a complexidade da solução é justificável pela flexibilidade e controle operacional que ela entrega.

6.1 Trabalhos Futuros

Este estudo focou primariamente na métrica de vazão (*throughput*) em operações de escrita sequencial. Para expandir a compreensão sobre o comportamento destas soluções em cenários ainda mais complexos, sugerem-se as seguintes linhas de pesquisa:

1. **Análise de IOPS de Escrita Aleatória:** Realizar *benchmarks* focados em operações de entrada e saída por segundo (IOPS) com blocos pequenos (4k), simulando cargas de trabalho de bancos de dados transacionais (OLTP), onde o comportamento do sistema de arquivos pode diferir drasticamente da escrita sequencial de grandes arquivos.

2. **Testes de Recuperação de Desastres (Chaos Engineering):** Avaliar o tempo de recuperação (*self-healing*) e o impacto no desempenho do cluster durante a falha forçada de um nó ou disco. Medir quanto tempo o CephFS e o GlusterFS levam para replicar novamente os dados perdidos e restaurar a redundância total.
3. **Comparativo com Longhorn e OpenEBS:** Estender o estudo para incluir soluções de armazenamento em bloco nativas de contêineres mais recentes, como o Longhorn (da SUSE) ou OpenEBS, que prometem simplificar a gestão do armazenamento distribuído no Kubernetes com menor *overhead* que o Ceph.

REFERÊNCIAS

Amazon Web Services. *Qual é a diferença entre os hipervisores tipo 1 e tipo 2?* 2024. Website. Disponível em: <<https://aws.amazon.com/pt/compare/the-difference-between-type-1-and-type-2-hypervisors/>>. Acesso em: 01 jul. 2025. 8

Cloud Native Computing Foundation. *CNCF Annual Survey 2023*. [S.l.], 2023. Relatório anual sobre o estado da adoção de tecnologias nativas da nuvem. Disponível em: <<https://www.cncf.io/reports/cncf-annual-survey-2023/>>. Acesso em: 03 fev. 2026. 3

Controle Net. *O que é NFS (Network File System)?* 2022. Disponível em: <<https://www.controle.net/faq/o-que-e-nfs-network-file-system>>. Acesso em: 07 ago. 2025. 27

Docker, Inc. *What is a Container?* 2025. O conteúdo original foi consultado a partir de um link traduzido. Disponível em: <<https://www.docker.com/resources/what-container/>>. Acesso em: 06 jul. 2025. 10, 11, 12, 13, 15

Flexera. *2024 State of the Cloud Report*. 2024. Relatório online. Análise anual sobre tendências de uso de nuvem e infraestrutura. Disponível em: <<https://info.flexera.com/CM-REPORT-State-of-the-Cloud>>. Acesso em: 03 fev. 2026. 3

FORNARI, F. et al. Distributed file systems performance tests on kubernetes/docker clusters. *Journal of Physics: Conference Series*, v. 2438, n. 012030, 2023. 29

Gluster Community. *GlusterFS Administrator Guide*. [S.l.], 2025. Disponível em: <<https://docs.gluster.org/en/latest/Administrator-Guide/>>. Acesso em: 24 nov. 2025. 57

GOMES, C. H. R. *CEPH COMO OBJECT STORAGE: Estudo de caso de implantação de hiperconvergência*. Monografia (Trabalho de Graduação (Tecnologia em Análise e Desenvolvimento de Sistemas)) — Faculdade de Tecnologia de Franca "Dr. Thomaz Novelino", Franca, SP, 2020. 29

- GOWRI, A. S. *Type 1 (bare-metal) and Type 2 (hosted) hypervisor*. 2019. Figura 7 do artigo: Impact of Virtualization Technologies in the Development and Management of Cloud Applications. Disponível em: <https://www.researchgate.net/figure/Type-1-bare-metal-and-Type-2-hosted-hypervisor_fig7_273119623>. Acesso em: 09 jul. 2025. 9
- IBM. *O que é virtualização?* 2024. Disponível em: <<https://www.ibm.com/br-pt/topics/virtualization>>. Acesso em: 06 jul. 2025. 6, 7, 9
- IBRYAM, B.; HUSS, R. *Padrões para Kubernetes: Desenvolvendo aplicações nativas para a nuvem de forma eficaz*. [S.l.]: Novatec Editora, 2020. ISBN 978-8575226939. 2
- Kubernetes. *Bootstrapping clusters with kubeadm*. 2025. Kubernetes Documentation. Ferramenta oficial para a criação e gerenciamento de clusters. Disponível em: <<https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/>>. Acesso em: 03 nov. 2025. 43
- Kubernetes. *Entendendo os objetos do Kubernetes*. 2025. Disponível em: <<https://kubernetes.io/pt-br/docs/concepts/overview/working-with-objects/kubernetes-objects/>>. Acesso em: 07 jul. 2025. 20
- Kubernetes. *Managing Swap Memory - Kubelet*. 2025. Kubernetes Documentation. Documentação oficial sobre desativação do swap. Disponível em: <<https://kubernetes.io/docs/concepts/architecture/nodes/#swap-memory>>. Acesso em: 03 nov. 2025. 41
- Kubernetes. *Volumes*. 2025. Disponível em: <<https://kubernetes.io/pt-br/docs/concepts/storage/volumes/>>. Acesso em: 07 jul. 2025. 21, 22, 27
- Kubernetes. *Volumes Persistentes*. 2025. Disponível em: <<https://kubernetes.io/pt-br/docs/concepts/storage/persistent-volumes/>>. Acesso em: 07 jul. 2025. 2, 4, 22, 23, 24, 26
- LIMA, C. S.; MEZZOMO, P. *Estudo sobre computação em nuvem e suas aplicações com Kubernetes*. Monografia (Trabalho de Conclusão de Curso (Tecnologia em Segurança da Informação)) — Faculdade de Tecnologia de Americana "Ministro Ralph Biasi", Americana, SP, 2022. 30
- LUKŠA, M. *Kubernetes in Action*. Second edition. [S.l.]: Manning, 2021. ISBN 978-1617296628. 2, 10, 16
- MAUERER, W. *Professional Linux Kernel Architecture*. [S.l.]: Wrox, 2021. Este livro aborda o KVM, base do Proxmox VE, detalhando sua operação como hipervisor Tipo 1. 8
- MIANO, L. P. *Análisis de modelos de acceso a volúmenes de datos mediante NFS en Kubernetes*. Dissertação (Trabajo Fin de Máster) — Universitat Politècnica de València, València, Espanha, 2024. 30
- Mirantis. *cri-dockerd - The bridge between Kubernetes CRI and the Docker Engine*. 2024. GitHub Repository. Adaptador para interação entre Kubelet e Docker Engine. Disponível em: <<https://github.com/Mirantis/cri-dockerd>>. Acesso em: 03 nov. 2025. 42

My Nettech. *Como o Docker funciona*. 2025. Disponível em: <<https://mynettech.com.br/docker/como-o-docker-funciona/>>. Acesso em: 07 ago. 2025. 14

PATEL, R. *Introduction to Kubernetes Persistent Volumes (PV) and Persistent Volume Claims (PVC)*. 2024. Disponível em: <<https://medium.com/@ravipatel.it/introduction-to-kubernetes-persistent-volumes-pv-and-persistent-volume-claims-pvc-2a7d0eff0a92>>. Acesso em: 07 ago. 2025. 25, 26

Project Calico. *Calico v3.26.1 Manifests*. 2024. GitHub. Manifesto de implantação para a versão 3.26.1 do Calico CNI. Disponível em: <<https://raw.githubusercontent.com/projectcalico/calico/v3.26.1/manifests/calico.yaml>>. Acesso em: 03 nov. 2025. 44

Proxmox Server Solutions GmbH. *Proxmox VE Documentation*. 2024. Disponível em: <<https://pve.proxmox.com/pve-docs/>>. Acesso em: 06 jul. 2025. 9

Red Hat, Inc. *O que são containers?* 2025. Atualizado em: 14 set. 2023. Disponível em: <<https://www.redhat.com/pt-br/topics/containers>>. Acesso em: 06 jul. 2025. 10, 11, 13

Rook Project. *Rook: Open-Source, Cloud-Native Storage for Kubernetes*. 2025. Disponível em: <<https://rook.io/docs/rook/latest/>>. Acesso em: 24 nov. 2025. 28, 61

SANTOS, L. *Kubernetes: Tudo sobre orquestração de contêineres*. [S.l.]: Casa do Código, 2019. ISBN 978-85-7254-135-2. 2, 13

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Operating System Concepts*. 10. ed. Hoboken, NJ: Wiley, 2018. 7

STALLINGS, W. *Operating Systems: Internals and Design Principles*. 9. ed. Harlow, United Kingdom: Pearson, 2018. 7

SUSNJARA, S. *A história do Kubernetes*. 2023. Publicado em: 02 nov. 2023. Disponível em: <<https://www.ibm.com/br-pt/think/topics/kubernetes-history>>. Acesso em: 07 jul. 2025. 17

TANENBAUM, A. S.; BOS, H. *Sistemas Operacionais Modernos*. 4. ed. São Paulo: Pearson Education do Brasil, 2016. 7, 10

The Gluster Community. *Gluster Documentation*. 2025. Disponível em: <<https://docs.gluster.org/en/latest/>>. Acesso em: 07 ago. 2025. 27

The Kubernetes. *Arquitetura do Kubernetes*. 2025. Disponível em: <<https://kubernetes.io/pt-br/docs/concepts/architecture/>>. Acesso em: 07 ago. 2025. 18

The Kubernetes. *Pod Lifecycle*. 2025. Documentação Oficial do Kubernetes. Disponível em: <<https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/>>. Acesso em: 29 jun. 2025. 1

VERMA, A. et al. Large-scale cluster management at google with Borg. In: *Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15)*. New York, NY, USA: ACM, 2015. p. 18:1–18:17. 17, 29

VUCIC, V. *O que é o armazenamento Ceph? Um guia para iniciantes*. 2021. Disponível em: <<https://phoenixnap.pt/gloss%C3%A1rio/o-que-%C3%A9-ceph>>. Acesso em: 07 ago. 2025. 28